

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC



TIỂU LUẬN MÔN HỌC
THIẾT KẾ VÀ ĐÁNH GIÁ THUẬT TOÁN

Tên đề tài:

**Tìm hiểu một số thuật toán cho bài toán
đề xuất bài hát từ dữ liệu hành vi người dùng**

Mã học phần: MAT3504

Giảng viên: PSG. TS ĐẶNG THỊ OANH

Sinh viên thực hiện:

Lê Bích Ngọc – 23000146

Nguyễn Thị Liên – 23000134

Nguyễn Văn Tuấn Anh – 23000090

Hà Nội, 2026

MỤC LỤC

BẢNG VÀ HÌNH

1	GIỚI THIỆU	1
1.1	Mục tiêu của đề tài	1
1.2	Phát biểu bài toán	1
2	CƠ SỞ LÝ THUYẾT	2
2.1	Các phương pháp/thuật toán sử dụng	2
2.1.1	Vết Cạn	2
2.1.2	Cắt Nhánh	3
2.1.3	Tham lam	4
2.1.4	Quy hoạch động	6
2.1.5	Heuristic	7
3	BÀI TOÁN ỨNG DỤNG	9
3.1	Thuật toán Vết cạn	9
3.1.1	Quy trình thực hiện	9
3.1.2	Thiết kế chương trình/hệ thống	9
3.1.3	Chi tiết cài đặt	10
3.2	Thuật toán Cắt Nhánh	11
3.2.1	Quy trình thực hiện	11
3.2.2	Thiết kế chương trình/hệ thống	11
3.2.3	Chi tiết cài đặt	11
3.3	Thuật toán Tham lam	13
3.3.1	Sự đánh đổi và mức độ đáp ứng hàm mục tiêu	13
3.3.2	Quy trình thực hiện tổng quát	13
3.3.3	Thiết kế chương trình và hệ thống	14
3.3.4	Chi tiết cài đặt và đánh giá độ phức tạp	14
3.4	Thuật toán caching kết hợp tiên xử lý	16
3.4.1	Sự đánh đổi và mức độ áp dụng lý thuyết quy hoạch động	16
3.4.2	Quy trình thực hiện tổng quát	16
3.4.3	Thiết kế chương trình và hệ thống	17
3.4.4	Chi tiết cài đặt và đánh giá độ phức tạp	17
3.5	Thuật toán Heuristic	19
3.5.1	Quy trình thực hiện	19
3.5.2	Thiết kế chương trình và hệ thống	19
3.5.3	Chi tiết cài đặt	20
4	KẾT QUẢ THỰC NGHIỆM	21
4.1	Môi trường thực nghiệm	21
4.2	Bộ dữ liệu và kịch bản thử nghiệm	22
4.3	Kết quả thực nghiệm	24
4.3.1	Thời gian thực thi	24
4.3.2	Tổng số phép tính	25
4.3.3	Sử dụng bộ nhớ	26
4.3.4	Khả năng mở rộng	26
4.3.5	Phân rã phép tính theo thành phần	27
4.3.6	Tỉ lệ thu hồi (Recovery Rate)	28

4.3.7	Hiệu năng bộ nhớ cache (CachingRecommender)	29
4.3.8	Hệ số tăng tốc so với BruteForce	30
4.4	Phân tích và so sánh kết quả	31
4.4.1	Bảng so sánh tổng hợp	31
4.4.2	Phân tích từng nhóm phương pháp	31
4.4.3	Phân tích đa chiều: Tương quan giữa tốc độ, chính xác và bộ nhớ	32
4.4.4	Nhận xét	33
5	KẾT LUẬN	33
5.1	Đánh giá các phương pháp	33
5.2	Ưu điểm và hạn chế	34
5.3	Hướng phát triển trong tương lai	34

BẢNG VÀ HÌNH

Danh sách hình vẽ

1	Sơ đồ lớp kiến trúc của hệ thống gợi ý âm nhạc	23
2	Thời gian thực thi của các thuật toán theo kịch bản (ms)	24
3	Tổng số phép tính của các thuật toán theo kịch bản	25
4	Bộ nhớ phụ sử dụng của các thuật toán theo kịch bản (KB)	26
5	Khả năng mở rộng: thời gian và số phép tính theo kích thước đầu vào N	26
6	Phân rã số phép tính (Similarity Ops, Heap Ops, Score Ops) - Kịch bản Large (N=500)	27
7	Tỉ lệ thu hồi so với BruteForce (%) - Ma trận thuật toán $\times$ kịch bản	28
8	Tỉ lệ cache hit/miss và so sánh thời gian V1 TopDownMemo vs V2 BottomUpTable	29
9	Hệ số tăng tốc so với BruteForce - Kịch bản Large (N=500, BruteForce = 643.246 ms)	30

Danh sách bảng

1	Đánh đổi tốc độ - recovery ở kịch bản Large (N=500)	29
2	So sánh tổng hợp tám thuật toán - Kịch bản Large (N=500, K=5)	31
3	Khuyến nghị thuật toán theo yêu cầu ứng dụng	33

1 GIỚI THIỆU

1.1 Mục tiêu của đề tài

Trong thời đại bùng nổ thông tin, số lượng sản phẩm âm nhạc trên các nền tảng trực tuyến đang gia tăng theo cấp số nhân. Điều này dẫn đến bài toán quá tải thông tin, khiến người dùng gặp nhiều khó khăn trong việc tìm kiếm những bài hát phù hợp với sở thích cá nhân. Để giải quyết vấn đề này, hệ thống gợi ý đã trở thành một công cụ giúp cá nhân hóa trải nghiệm và tối ưu hóa luồng phân phối nội dung. Đề tài này được thực hiện nhằm giải quyết bài toán gợi ý âm nhạc với hai mục tiêu chính: mục tiêu ứng dụng thực tiễn và mục tiêu cốt lõi về phân tích thuật toán. Trong khuôn khổ một tiểu luận môn học, đề tài không hướng tới việc triển khai các mô hình học máy phức tạp như các hệ thống công nghiệp hiện hành, mà tập trung vào việc mô hình hóa bài toán để thử nghiệm và đánh giá các thuật toán cơ bản đã được học.

Đề tài này tập trung xây dựng một hệ thống gợi ý âm nhạc dựa trên hành vi nghe nhạc của người dùng. Dữ liệu được sử dụng là tập Last.fm 1K Dataset, trong đó lịch sử nghe nhạc của người dùng được xử lý và chuyển thành các trọng số tương tác giữa người dùng và bài hát. Những trọng số này được dùng để xác định mức độ yêu thích của người dùng đối với từng bài hát.

Bên cạnh ý nghĩa ứng dụng, mục tiêu chính của đề tài là nghiên cứu và đánh giá các chiến lược thuật toán khác nhau trong bài toán gợi ý. Thay vì tập trung vào các mô hình học máy phức tạp, đề tài tiếp cận bài toán dưới góc độ của môn học Thiết kế và đánh giá thuật toán. Trọng tâm không chỉ là chất lượng gợi ý mà còn là hiệu năng xử lý khi dữ liệu có kích thước lớn và phân bố thưa. Cụ thể, đề tài tiến hành cài đặt, phân tích độ phức tạp và đánh giá thực nghiệm năm chiến lược thuật toán khác nhau: thuật toán Vết cạn làm cơ sở đánh giá độ chính xác tuyệt đối, thuật toán tham lam tối ưu hóa cục bộ, thuật toán nhánh cận để ngắt sớm không gian tìm kiếm, thuật toán quy hoạch động đánh đổi không gian lưu trữ lấy thời gian truy vấn, và cuối cùng là thuật toán Heuristic kết hợp cấu trúc chỉ mục ngược nhằm tận dụng triệt để đặc tính thưa thớt của dữ liệu. Tuy nhiên, cần lưu ý rằng cách áp dụng các thuật toán ở đây chủ yếu lấy ý tưởng và biến đổi cho phù hợp với bài toán và dữ liệu.

Thông qua việc đối chiếu các phương pháp này, đề tài minh chứng cho sự đánh đổi giữa thời gian chạy, tài nguyên bộ nhớ và độ chính xác của tập kết quả (th nghiệm tối ưu toàn cục so với nghiệm xấp xỉ). Kết quả của bài toán sau khi áp dụng các thuật toán tương ứng sẽ được đánh giá chi tiết trong phần thực nghiệm nhằm làm rõ sự ưu/nhược điểm của từng phương pháp.

1.2 Phát biểu bài toán

Bài toán gợi ý âm nhạc trong đề tài được mô hình hóa dưới dạng bài toán tối ưu hóa tìm kiếm Top- K phần tử trên một không gian dữ liệu lớn. Hệ thống bao gồm một tập hợp người dùng $U = \{u_1, u_2, \dots, u_n\}$ và một tập hợp bài hát $S = \{s_1, s_2, \dots, s_m\}$. Dữ liệu tương tác giữa người dùng và bài hát được biểu diễn bằng một ma trận thưa $R \in \mathbb{R}^{n \times m}$, trong đó mỗi phần tử $R_{u,s}$ thể hiện số lần người dùng u đã nghe bài hát s . Đặc trưng của ma trận này là tính thưa thớt cực cao, do phần lớn người dùng u chưa từng nghe bài hát s , tức là $R_{u,s} = 0$.

Cho trước một người dùng mục tiêu $u_t \in U$ và số lượng bài hát cần gợi ý K . Gọi $S_{u_t} = \{s \in S \mid R_{u_t,s} > 0\}$ là tập hợp các bài hát mà u_t đã từng nghe. Hệ thống cần tìm kiếm các gợi ý từ tập ứng viên tiềm năng $C = S \setminus S_{u_t}$, đảm bảo không gợi ý lại những bài hát cũ.

Quá trình giải quyết bài toán được chia làm hai pha tính toán. Pha thứ nhất là xác định mức độ tương đồng giữa người dùng mục tiêu u_t và các người dùng khác trong hệ thống. đề tài sử dụng độ đo tương đồng (cosine similarity) trên vector đặc trưng của người dùng. Với mọi người dùng $v \in U \setminus \{u_t\}$,

độ tương đồng được tính bằng công thức:

$$\text{sim}(u_t, v) = \frac{\sum_{s \in S_{u_t} \cap S_v} R_{u_t, s} \cdot R_{v, s}}{\sqrt{\sum_{s \in S_{u_t}} R_{u_t, s}^2} \times \sqrt{\sum_{s \in S_v} R_{v, s}^2}} \quad (1)$$

Trong đó, tử số là tích vô hướng các bài hát chung giữa hai người dùng, và mẫu số là tích độ dài của hai vector. Dựa trên kết quả này, ta định nghĩa tập hợp các láng giềng có ý nghĩa $N(u_t) = \{v \in U \mid \text{sim}(u_t, v) > 0\}$.

Pha thứ hai là chấm điểm (scoring) cho các bài hát thuộc tập ứng viên C . Điểm số của một bài hát $s \in C$ đối với người dùng u_t được tổng hợp từ lịch sử nghe nhạc của các láng giềng, trọng số hóa bởi độ tương đồng của họ:

$$\text{Score}(u_t, s) = \sum_{v \in N(u_t) \mid R_{v, s} > 0} \text{sim}(u_t, v) \times R_{v, s} \quad (2)$$

Về mặt lý thuyết toán học, mục tiêu tuyệt đối của hệ thống là tìm ra một tập con tối ưu toàn cục $S^* \subset C$ thỏa mãn kích thước $|S^*| = K$, sao cho tổng điểm của các bài hát đạt giá trị cực đại:

$$S^* = \arg \max_{\substack{S' \subset C \\ |S'| = K}} \sum_{s \in S'} \text{Score}(u_t, s) \quad (3)$$

Tuy nhiên, thách thức của bài toán này nằm ở độ phức tạp tính toán. Nếu tiếp cận theo phương pháp quét toàn bộ (vét cạn) để chắc chắn tìm ra S^* , chi phí thời gian cho mỗi lượt truy vấn sẽ đạt mức $\mathcal{O}(|U| \cdot |S|)$. Điều này hoàn toàn bất khả thi đối với các hệ thống thời gian thực yêu cầu độ trễ thấp.

Do đó, bài toán thử nghiệm các chiến lược thuật toán khác nhau. Việc này dẫn đến một sự đánh đổi: trong khi một số thuật toán cố gắng duy trì tính toàn vẹn để tìm ra đúng S^* , thì một số thuật toán khác (như tham lam hay Heuristic) chủ động nói lỏng phương trình (3), chấp nhận tìm kiếm một tập nghiệm xấp xỉ $\hat{S}$ (nghiệm tối ưu cục bộ). Mục đích của sự nói lỏng này là để cực tiểu hóa số lượng phép tính, đánh đổi một phần nhỏ độ chính xác của tập kết quả nhằm tối ưu tốc độ phản hồi.

2 CƠ SỞ LÝ THUYẾT

2.1 Các phương pháp/thuật toán sử dụng

2.1.1 Vét Cạn

Vét cạn là phương pháp giải quyết vấn đề bằng cách kiểm tra tất cả các khả năng hoặc giải pháp có thể có một cách có hệ thống cho đến khi tìm thấy một giải pháp đúng hoặc đã kiểm tra hết tất cả các trường hợp.

Mô hình của phương pháp Vét cạn:

Với $D = (x_1, x_2, \dots, x_n)$: Tập khả năng (Solution domain)

$x_i \in D_i$, $D_i = \{a_{ij} \mid j = 1 \dots m_i\}$ - tập hữu hạn.

Quy tắc xác định lời giải:

$f : D \rightarrow \{true, false\}$

Lời giải: $x = (x_1, x_2, \dots, x_n)$ sao cho $f(x) = true$

Listing 1: Mã giả thuật toán Vét Cạn

```
1 function BruteForce(problem):
```

```

2     bestSolution = null
3     bestValue = -infinity (or +infinity for minimization)
4
5     for each possible candidate in allPossibleSolutions(problem):
6         currentValue = evaluate(candidate)
7
8         if currentValue is better than bestValue:    // depending on max/min
9             bestValue = currentValue
10            bestSolution = candidate
11
12    return bestSolution, bestValue

```

Đánh giá thuật toán: Ưu, nhược điểm và độ phức tạp

Thuật toán Vét Cạn (Brute Force) có ưu điểm nổi bật là đảm bảo tìm được lời giải nếu lời giải tồn tại, miễn là có đủ thời gian và tài nguyên để duyệt toàn bộ không gian tìm kiếm. Đây là đặc điểm quan trọng nhất khiến phương pháp này thường được sử dụng làm chuẩn đối chiếu khi đánh giá tính đúng đắn của các thuật toán khác. Bên cạnh đó, thuật toán có nguyên lý hoạt động đơn giản, dễ hiểu và dễ triển khai, không đòi hỏi các kỹ thuật hay cấu trúc dữ liệu phức tạp. Nhờ tính tổng quát cao, phương pháp này có thể áp dụng cho nhiều dạng bài toán khác nhau mà không cần thay đổi đáng kể về cách tiếp cận.

Tuy nhiên, thuật toán Vét Cạn cũng tồn tại nhiều hạn chế. Nhược điểm lớn nhất là hiệu suất thấp do phải kiểm tra toàn bộ các khả năng có thể xảy ra, khiến độ phức tạp thời gian thường tăng theo hàm mũ hoặc giai thừa khi kích thước bài toán tăng lên. Điều này làm cho phương pháp trở nên không khả thi đối với các bài toán có không gian tìm kiếm lớn. Ngoài ra, việc duyệt toàn bộ các trường hợp cũng tiêu tốn đáng kể tài nguyên tính toán như bộ nhớ và CPU. Trong các bài toán tối ưu hóa, mặc dù thuật toán có thể tìm được lời giải, nhưng không phải lúc nào cũng mang lại hiệu quả tốt về mặt thời gian xử lý so với các phương pháp tối ưu hơn được thiết kế chuyên biệt cho từng loại bài toán.

2.1.2 Cắt Nhánh

Cắt nhánh (Pruning) là phương pháp cải tiến từ Vét Cạn, trong đó loại bỏ sớm các nhánh không có khả năng dẫn đến lời giải tốt hơn lời giải tốt nhất hiện tại, nhờ đó giảm đáng kể không gian tìm kiếm cần duyệt.

Mô hình của phương pháp Cắt Nhánh: Với $D = (x_1, x_2, \dots, x_n)$: Tập khả năng (Solution domain)

$$x_i \in D_i, \quad D_i = \{a_{ij} \mid j = 1 \dots m_i\} \text{ - tập hữu hạn.}$$

Hàm cận trên (upper bound) cho nhánh tại bước i : $g : D_1 \times \dots \times D_i \rightarrow \mathbb{R}$

Quy tắc cắt nhánh:

Nếu $g(x_1, \dots, x_i) \leq \text{bestValue}$ thì loại bỏ toàn bộ nhánh $(x_1, \dots, x_i, *, \dots, *)$

Lời giải: $x = (x_1, x_2, \dots, x_n)$ sao cho $f(x) = \text{true}$ và $f(x)$ là tốt nhất

Listing 2: Mã giả thuật toán Cắt Nhánh

```

1 function Pruning(problem):
2     bestSolution = null
3     bestValue = -infinity (or +infinity for minimization)
4
5     for each possible candidate in allPossibleSolutions(problem):
6         upperBound = estimateUpperBound(candidate)
7
8         if upperBound <= bestValue:    // pruning condition
9             skip this candidate and its descendants
10            continue

```

```
11     currentValue = evaluate(candidate)
12
13
14     if currentValue is better than bestValue:
15         bestValue = currentValue
16         bestSolution = candidate
17
18     return bestSolution, bestValue
```

Đánh giá thuật toán: Ưu, nhược điểm và độ phức tạp

Thuật toán Cắt Nhánh có ưu điểm nổi bật là giảm đáng kể không gian tìm kiếm so với Vét Cạn bằng cách loại bỏ sớm các nhánh không tiềm năng thông qua hàm cận. Trong trường hợp tốt, thuật toán có thể bỏ qua phần lớn các ứng viên mà không cần đánh giá đầy đủ, giúp tăng tốc độ xử lý đáng kể. Phương pháp này vẫn đảm bảo tính đúng đắn nếu hàm cận được xây dựng chính xác — tức là hàm cận không được đánh giá thấp hơn giá trị thực của bất kỳ nghiệm nào trong nhánh bị cắt.

Tuy nhiên, thuật toán Cắt Nhánh cũng tồn tại một số hạn chế. Hiệu quả của thuật toán phụ thuộc hoàn toàn vào chất lượng của hàm cận: nếu hàm cận quá lỏng (overestimate nhiều), rất ít nhánh bị cắt và thuật toán gần như tương đương Vét Cạn; ngược lại, nếu hàm cận không chính xác có thể dẫn đến cắt nhầm nhánh chứa nghiệm tối ưu, làm sai kết quả. Ngoài ra, việc xây dựng hàm cận tốt đòi hỏi hiểu sâu về cấu trúc bài toán, làm tăng độ phức tạp khi triển khai. Độ phức tạp thời gian trong trường hợp xấu nhất vẫn tương đương Vét Cạn, tuy nhiên trong thực tế thường nhanh hơn đáng kể nhờ cơ chế cắt nhánh hiệu quả.

2.1.3 Tham lam

Định nghĩa

Thuật toán tham lam là một phương pháp thiết kế thuật toán theo tư duy xây dựng nghiệm từng bước. Tại mỗi bước, thuật toán luôn đưa ra một lựa chọn tốt nhất tại thời điểm hiện tại (tối ưu cục bộ) với hy vọng rằng chuỗi các lựa chọn này sẽ dẫn đến kết quả tốt nhất cho toàn bộ bài toán (tối ưu toàn cục). Đặc trưng cốt lõi của phương pháp này là tính "không thể đảo ngược": một khi một quyết định đã được đưa ra, thuật toán sẽ không bao giờ xem xét lại hay thay đổi quyết định đó trong các bước tương lai.

Các thành phần cơ bản

Một thuật toán tham lam tiêu chuẩn thường được thiết kế dựa trên năm thành phần cốt lõi có mối quan hệ mật thiết với nhau. Thuật toán bắt đầu với một tập ứng viên chứa tất cả các phần tử đầu vào có thể sử dụng để xây dựng nghiệm. Tại mỗi bước lặp, một hàm lựa chọn sẽ được gọi để đánh giá và trích xuất ứng viên tốt nhất tiếp theo từ tập ứng viên. Ngay sau đó, một hàm khả thi sẽ kiểm tra xem việc đưa ứng viên này vào tập kết quả hiện tại có vi phạm các ràng buộc của bài toán hay không. Xuyên suốt quá trình này, một hàm mục tiêu được sử dụng để định lượng giá trị (chi phí hoặc lợi nhuận) của nghiệm đang được xây dựng. Cuối cùng, một hàm giải pháp sẽ liên tục kiểm tra xem cấu trúc hiện tại đã đáp ứng đủ các điều kiện để trở thành một nghiệm hoàn chỉnh của bài toán hay chưa.

Điều kiện áp dụng

Không phải bài toán tối ưu nào cũng có thể giải được bằng thuật toán tham lam. Để phương pháp này đảm bảo tìm ra nghiệm tối ưu toàn cục, bài toán bắt buộc phải thỏa mãn hai tính chất lý thuyết toán học sau:

Đầu tiên là tính chất lựa chọn tham lam. Một nghiệm tối ưu toàn cục có thể đạt được bằng cách thực hiện các lựa chọn tối ưu cục bộ. Điểm khác biệt lớn nhất so với Quy hoạch động là thuật toán tham lam có thể đưa ra lựa chọn ngay lập tức dựa trên thông tin hiện tại mà không cần phải giải quyết các bài toán con trước.

Tiếp theo là cấu trúc con tối ưu. Bài toán phải có tính chất cấu trúc con tối ưu, nghĩa là một nghiệm tối ưu của bài toán ban đầu luôn chứa đựng các nghiệm tối ưu của các bài toán con bên trong nó.

Đánh giá thuật toán: Ưu, nhược điểm và độ phức tạp

Điểm mạnh lớn nhất của thuật toán tham lam là sự trực quan, dễ tiếp cận và dễ cài đặt. Nhờ việc không lưu trữ hay xét lại các trạng thái cũ, bộ nhớ phụ trợ thường rất nhỏ. Đặc biệt, nó mang lại tốc độ thực thi rất nhanh, do đó thường được sử dụng làm thuật toán xấp xỉ hữu hiệu cho các bài toán $\mathcal{NP}$ -Khó (như bài toán TSP).

Rủi ro lớn nhất của phương pháp này là dễ rơi vào "bẫy" tối ưu cục bộ và không thể tìm được nghiệm tối ưu toàn cục nếu bài toán không thỏa mãn tính chất lựa chọn tham lam. Do đó, khi thiết kế thuật toán tham lam, cần phải chứng minh được tính đúng đắn toán học của nó.

Tốc độ của thuật toán tham lam phụ thuộc chủ yếu vào cách cài đặt hàm lựa chọn. Thông thường, để lấy ra phần tử tốt nhất ở mỗi bước, thuật toán yêu cầu phải sắp xếp tập ứng viên từ trước, dẫn đến độ phức tạp cơ sở là $\mathcal{O}(N \log N)$. Nếu sử dụng các cấu trúc dữ liệu nâng cao như hàng đợi ưu tiên (Priority Queue / Min-Max Heap), chi phí trích xuất phần tử có thể giảm xuống còn $\mathcal{O}(\log N)$ ở mỗi bước lặp, giúp hệ thống hoạt động vô cùng hiệu quả với dữ liệu lớn.

Mã giả

Dưới đây là cấu trúc mã giả mô tả khung thuật toán tham lam tổng quát cho một bài toán tối ưu hóa:

Listing 3: Mã giả tổng quát của Thuật toán Tham lam

```

1 function Greedy(C: Set_of_Candidates) returns Set_of_Candidates
2   // C là tập hợp tất cả các ứng viên ban đầu
3   Sol := {} // Khởi tạo giải pháp rỗng
4
5   while (not Solution(Sol)) and (C != {}) do
6     x := Select(C) // Chọn ứng viên tốt nhất hiện tại từ C
7     C := C \ {x}   // Loại bỏ ứng viên x khỏi tập C
8
9     if Feasible(Sol U {x}) then
10      Sol := Sol U {x} // Thêm x vào giải pháp nếu khả thi
11    end if
12  end while
13
14  if Solution(Sol) then
15    return Sol // Trả về giải pháp hoàn chỉnh
16  else
17    return "No solution found" // Không tìm thấy giải pháp hợp lệ
18  endif
19 end function

```

Trong đó, các khối logic cốt lõi được định nghĩa cụ thể:

- **Select(C)**: Đại diện cho hàm lựa chọn tiêu chí tham lam, trả về phần tử có tiềm năng mang lại lợi ích lớn nhất trong tập C .
- **Feasible(Sol $\cup$ {x})**: Đánh giá tính hợp lệ của cấu trúc tập kết quả nếu tiếp nạp thêm phần tử x .
- **Solution(Sol)**: Hàm xác nhận tập kết quả đã đạt điều kiện dừng (kết thúc bài toán) hay chưa.

2.1.4 Quy hoạch động

Định nghĩa và bản chất

Quy hoạch động là một phương pháp thiết kế thuật toán tối ưu hóa mạnh mẽ, được khởi xướng bởi nhà toán học Richard Bellman, chuyên dùng để giải quyết các bài toán phức tạp bằng cách chia nhỏ chúng thành các bài toán con đơn giản hơn. Khác với những phương pháp duyệt ngây thơ, bản chất cốt lõi của quy hoạch động nằm ở cơ chế lưu trữ và tái sử dụng. Thay vì phải tính toán lại từ đầu mỗi khi gặp một bài toán con quen thuộc, thuật toán sẽ lưu kết quả của nó vào một cấu trúc dữ liệu (mảng, ma trận hoặc bảng băm). Khi bài toán con đó xuất hiện lại, hệ thống chỉ cần truy xuất kết quả đã lưu trong thời gian $\mathcal{O}(1)$. Cơ chế này giúp triệt tiêu các nhánh tính toán dư thừa, thường đưa độ phức tạp thời gian của bài toán từ hàm mũ $\mathcal{O}(2^n)$ hoặc giai thừa xuống mức hàm đa thức $\mathcal{O}(n^k)$.

Điều kiện áp dụng cốt lõi

Để một bài toán có thể giải quyết hiệu quả bằng quy hoạch động, cấu trúc toán học của bài toán đó bắt buộc phải thỏa mãn hai tính chất nền tảng: Thứ nhất là cấu trúc con tối ưu. Theo nguyên lý tối ưu của Bellman, một bài toán có cấu trúc con tối ưu nếu giải pháp tối ưu của bài toán gốc được cấu thành từ giải pháp tối ưu của các bài toán con bên trong nó. Điều này cho phép thiết lập một công thức truy hồi để liên kết các trạng thái với nhau.

Thứ hai là các bài toán con gối nhau. Đây là điểm ranh giới phân biệt rõ nhất giữa quy hoạch động và kỹ thuật chia để trị. Trong khi chia để trị phân rã bài toán thành các phần hoàn toàn độc lập (như Merge Sort), thì quy hoạch động được sinh ra để xử lý các bài toán mà quá trình chia nhỏ tạo ra một lượng lớn các bài toán con trùng lặp, bị gọi đi gọi lại nhiều lần trong không gian trạng thái.

Cài đặt và đánh giá độ phức tạp

Tùy thuộc vào luồng thực thi và đặc thù của đồ thị trạng thái, quy hoạch động thường được triển khai theo hai phương pháp tiếp cận chính, mỗi phương pháp đều có sự đánh đổi riêng về tài nguyên:

Thứ nhất là Top-down (Đệ quy có nhớ - Memoization). Thuật toán tiếp cận bài toán theo chiều từ trên xuống, bắt đầu từ trạng thái gốc và gọi đệ quy xuống các bài toán con. Trước khi xử lý, thuật toán kiểm tra trong bộ nhớ đệm xem trạng thái này đã được giải quyết chưa. Điểm mạnh của Top-down là tính tự nhiên trong tư duy và chỉ tính toán những trạng thái thực sự cần thiết. Tuy nhiên, phương pháp này phải chịu chi phí hao tổn từ Call Stack của đệ quy, có rủi ro gây tràn bộ nhớ nếu độ sâu quá lớn.

Thứ hai là Bottom-up (Lập bảng phương án - Tabulation). Tiếp cận bài toán theo chiều từ dưới lên, giải quyết các bài toán con cơ sở (nhỏ nhất) trước và điền kết quả vào một bảng phương án theo đúng thứ tự tô-pô. Các bài toán lớn hơn sẽ được tính dựa trên các ô đã điền trước đó. Phương pháp này hoàn toàn sử dụng vòng lặp, triệt tiêu rủi ro tràn Stack và tốc độ thực thi thực tế thường nhanh hơn do tận dụng được bộ nhớ đệm của CPU. Hơn nữa, nó cho phép áp dụng các kỹ thuật tối ưu hóa không gian để giảm thiểu tài nguyên lưu trữ.

Các bước thiết kế thuật toán

Quy trình thiết kế một giải pháp quy hoạch động chuẩn mực thường tuân thủ nghiêm ngặt 4 bước sau:

1. Xác định cấu trúc của giải pháp tối ưu: Định nghĩa rõ ràng bài toán con và tìm ra các biến trạng thái (tham số) đặc trưng cho một tình huống cụ thể của bài toán.
2. Thiết lập công thức truy hồi (Hàm chuyển trạng thái): Biểu diễn toán học giá trị của trạng thái hiện tại thông qua giá trị của các trạng thái nhỏ hơn. Đồng thời, xác định rõ các trường hợp cơ sở làm điểm dừng.
3. Tính toán giá trị giải pháp tối ưu: Chọn cách tiếp cận (thường là Bottom-up) để khởi tạo và lấp đầy bảng phương án.

4. Truy vết tìm nghiệm: Sử dụng bảng phương án đã xây dựng ở bước 3 để lùi ngược lại, xác định chính xác trình tự các lựa chọn đã tạo ra giá trị tối ưu đó (từ đó cấu trúc lại nghiệm của bài toán).

Mã giả

Dưới đây là khung mã giả tổng quát minh họa cho hai chiến lược triển khai Top-down và Bottom-up của Quy hoạch động:

Listing 4: Cách tiếp cận Top-down (Memoization)

```

1 // Khởi tạo mảng memo với giá trị rỗng hoặc sentinel value (vd: -1, NaN)
2 Array memo[1..N]
3
4 function DP_TopDown(state):
5     // 1. Trường hợp cơ sở (Base case)
6     if state is BaseCase then
7         return BaseValue
8
9     // 2. Kiểm tra xem bài toán con đã được giải quyết chưa
10    if memo[state] != -1 then
11        return memo[state]
12
13    // 3. Tính toán thông qua công thức truy hồi
14    result := Calculate( DP_TopDown(sub_state_1), DP_TopDown(sub_state_2), ... )
15
16    // 4. Lưu vào bảng Cache và trả về kết quả
17    memo[state] := result
18    return result
19 end function

```

Listing 5: Cách tiếp cận Bottom-up (Tabulation)

```

1 function DP_BottomUp(N):
2     // 1. Khởi tạo bảng phương án (ví dụ mảng 1 chiều)
3     Array dp[0..N]
4
5     // 2. Thiết lập các trường hợp cơ sở (Base cases)
6     dp[0] := BaseValue_0
7     dp[1] := BaseValue_1
8     // ...
9
10    // 3. Xây dựng bảng từ dưới lên (theo thứ tự To-po)
11    for i := 2 to N do
12        dp[i] := Calculate( dp[i-1], dp[i-2], ... )
13    end for
14
15    // 4. Trả về kết quả của bài toán gốc
16    return dp[N]
17 end function

```

2.1.5 Heuristic

Bản chất

Heuristic là nhóm phương pháp thiết kế thuật toán dựa trên các quy tắc kinh nghiệm hoặc tri thức miền

nhằm giảm chi phí tính toán bằng cách thu hẹp không gian tìm kiếm. Khác với các thuật toán chính xác, heuristic không đảm bảo luôn tìm được nghiệm tối ưu toàn cục, mà ưu tiên đạt được lời giải đủ tốt trong thời gian hợp lý.

Bản chất của heuristic nằm ở việc tận dụng các đặc điểm thực nghiệm của dữ liệu để loại bỏ sớm những trường hợp ít tiềm năng, từ đó giảm đáng kể số phép tính cần thực hiện.

Trong hệ gợi ý bài hát, heuristic được sử dụng thuộc nhóm **heuristic lọc ứng viên** (candidate filtering heuristic). Ý tưởng chính là chỉ thực hiện tính toán similarity trên những cặp người dùng có khả năng thực sự tương đồng, thay vì duyệt toàn bộ như phương pháp vét cạn.

Đặc trưng

Heuristic lọc ứng viên trong bài toán gợi ý bài hát dựa trên các quan sát thực nghiệm sau:

- Hai người dùng có rất ít bài hát chung thường có cosine similarity rất thấp hoặc bằng 0.
- Trong dữ liệu thực tế, ma trận tương tác người dùng – bài hát thường rất thưa (sparse), khiến phần lớn các cặp người dùng không có giao nhau đáng kể.
- Phần lớn thời gian xử lý của phương pháp vét cạn bị tiêu tốn cho các phép tính similarity không mang lại giá trị.

Từ đó, heuristic được xây dựng theo nguyên tắc:

Chỉ những người dùng có đủ số lượng bài hát chung mới được đưa vào tập ứng viên để tính similarity.

Cơ chế lọc ứng viên

Cho người dùng u và người dùng v , gọi:

$$\text{overlap}(u, v) \quad (4)$$

là số bài hát xuất hiện trong cả hai tập nghe nhạc. Ngưỡng lọc được xác định:

$$\text{threshold}(u, v) = \alpha \times \min(|S_u|, |S_v|) \quad (5)$$

trong đó:

- S_u, S_v là tập bài hát của người dùng u và v
- α là hệ số lọc

Một người dùng v chỉ được đưa vào tập ứng viên nếu:

$$\text{overlap}(u, v) \geq \text{threshold}(u, v) \quad (6)$$

Ngược lại, cặp người dùng đó sẽ bị loại bỏ trước khi thực hiện tính cosine similarity.

Ưu điểm

- Giảm mạnh số phép tính: Phần lớn các cặp người dùng bị loại bỏ ngay từ bước lọc, giúp giảm đáng kể số lần tính similarity.
- Khai thác đúng đặc điểm dữ liệu thực tế: Heuristic hoạt động đặc biệt hiệu quả trên dữ liệu sparse.
- Dễ điều chỉnh: Tham số α cho phép kiểm soát mức độ đánh đổi giữa tốc độ và độ bao phủ kết quả.
- Không thay đổi độ đo similarity: Thuật toán vẫn sử dụng cosine similarity chính xác trên tập ứng viên còn lại.

Nhược điểm

- Không đảm bảo tối ưu toàn cục: Một số người dùng tiềm năng có thể bị loại bỏ sớm nếu chưa đạt ngưỡng overlap.
- Phụ thuộc vào tham số α : Nếu α quá lớn, recall giảm mạnh; nếu quá nhỏ, hiệu quả lọc không đáng kể.
- Chi phí tiền xử lý: Một số phiên bản heuristic cần xây dựng thêm inverted index hoặc cấu trúc dữ liệu hỗ trợ lọc ứng viên.

3 BÀI TOÁN ỨNG DỤNG

3.1 Thuật toán Vết cạn

3.1.1 Quy trình thực hiện

Thuật toán Vết Cạn gợi ý bài hát theo phương pháp lọc cộng tác (Collaborative Filtering) dựa trên độ tương đồng cosine giữa người dùng mục tiêu và toàn bộ người dùng còn lại trong hệ thống, đồng thời hỗ trợ đo lường chi tiết hiệu suất để so sánh sau này.

Quy trình thực hiện: Lấy lịch sử user $\rightarrow$ Vết cạn tính similarity với mọi user khác $\rightarrow$ Dùng similarity để tính điểm dự đoán cho mọi bài hát $\rightarrow$ Dùng Min-Heap chọn ra Top k bài hát chưa nghe có điểm cao nhất $\rightarrow$ Trả về danh sách gợi ý.

Điểm đặc trưng của thuật toán này là tính đơn giản, dễ triển khai và luôn đảm bảo kết quả tối ưu (không bỏ sót bất kỳ người dùng lảng giềng nào). Tuy nhiên, độ phức tạp thời gian là $O(N)$ cho bước tính similarity, khiến hiệu năng giảm đáng kể khi tập dữ liệu lớn.

3.1.2 Thiết kế chương trình/hệ thống

Cấu trúc lớp BruteForceRecommender:

- data: Map<User, Map<Song, Interaction> — dữ liệu gốc.
- Các bộ đếm opCountSimilarity, opCountHeap, opCountScore, extraMemoryBytes phục vụ benchmark.

Luồng xử lý:

1. **Lấy lịch sử nghe nhạc:** Truy xuất tập bài hát mà người dùng đích (`target user`) đã nghe (`userHistory`) từ cơ sở dữ liệu tương tác.
2. **Tính độ tương đồng với tất cả người dùng:** Duyệt qua toàn bộ N người dùng khác trong hệ thống. Với mỗi người dùng, tính cosine similarity giữa vector tương tác của họ và vector của người dùng đích. Kết quả được lưu vào tập `neighbours`.
3. **Tính điểm dự đoán cho từng bài hát:** Dựa trên tập `neighbours` và mức độ tương đồng tương ứng, hàm `Scoring.score()` tổng hợp điểm dự đoán cho mỗi bài hát chưa được nghe.
4. **Lấy top-K bài hát:** Sử dụng min-heap kích thước k để duy trì k bài hát có điểm cao nhất. Kết quả cuối cùng được sắp xếp theo thứ tự điểm giảm dần và trả về.

3.1.3 Chi tiết cài đặt

Đoạn mã sau mô tả phương thức gợi ý bài hát theo phương pháp Vết Cạn:

```

1 function recommend(user, k):
2     // Step 1: Get the user's listening history
3     userHistory = data.get(user) or empty map
4
5     // Step 2: Find similar users (neighbors) using cosine similarity
6     neighbours = empty map
7     for each otherUser in data.keys:
8         if otherUser != user:
9             opCountSimilarity = opCountSimilarity + 1
10            similarity = Similarity.cosine(userHistory, data.get(otherUser))
11            neighbours[otherUser] = similarity
12
13    // Step 3: Compute recommendation scores for songs
14    scores = Scoring.score(user, data, neighbours)
15
16    if scores is empty:
17        return empty list
18
19    opCountScore = opCountScore + size of scores
20
21    // Step 4: Use a min-heap (Priority Queue) to get top-k songs
22    minHeap = new PriorityQueue() // sorted by score (ascending)
23
24    for each (song, score) in scores:
25        if userHistory contains song: // skip songs user already heard
26            continue
27
28        minHeap.offer( (song, score) )
29        opCountHeap = opCountHeap + 1
30
31        if minHeap.size > k:
32            minHeap.poll() // remove worst song
33            opCountHeap = opCountHeap + 1
34
35    // Step 5: Track extra memory usage (for analysis)
36    extraMemoryBytes += (neighbours.size * 64)
37                      + (scores.size * 64)
38                      + ((k + 1) * 32)
39
40    // Step 6: Extract results from heap (in descending order)
41    result = empty list
42    while minHeap is not empty:
43        song = minHeap.poll().key
44        result.add_to_front(song) // insert at beginning to reverse order
45
46    return result

```

Phân tích độ phức tạp:

- Độ phức tạp thời gian: $O(N \cdot M + S \cdot \log k)$ — với N là số người dùng, M là số bài hát trung bình mỗi người dùng, S là số bài ứng viên, k là số bài cần gợi ý. Thuật toán duyệt toàn bộ N người dùng để tính cosine similarity ($O(N \cdot M)$), sau đó tính điểm ứng viên ($O(N \cdot M)$) và duy trì heap top- k ($O(S \cdot \log k)$).
- Độ phức tạp không gian: $O(N + S + k)$ phụ trợ mỗi truy vấn — lưu bảng neighbours ($O(N)$), bảng điểm scores ($O(S)$) và heap kết quả ($O(k)$).

3.2 Thuật toán Cắt Nhánh

3.2.1 Quy trình thực hiện

Thuật toán Cắt Nhánh vẫn sử dụng Collaborative Filtering như Vết Cạn, nhưng không tính cosine similarity một cách mù quáng với tất cả người dùng.

Sử dụng kỹ thuật Pruning (Cạn) + Upper Bound (Giới hạn trên) để loại sớm (prune) những user chắc chắn không nằm trong top k neighbours tương đồng nhất → tiết kiệm rất nhiều phép tính.

3.2.2 Thiết kế chương trình/hệ thống

Cấu trúc lớp PruningRecommender:

- `data` : `Map<User, Map<Song, Interaction>` — dữ liệu gốc.
- `norm` : `Map<User, Double>` — độ dài vector của mỗi user.
- `sortedEntries` : `Map<User, List<Map.Entry<Song, Interaction>>` — Danh sách bài hát đã sắp xếp giảm dần theo `playCount` của mỗi user.
- `suffixSqSum` : `Map<User, double[]>` — Mảng tổng bình phương suffix (từ vị trí i trở về sau).

Luồng xử lý:

1. Tính trước thông tin hỗ trợ pruning cho tất cả user.
2. Duyệt tất cả user khác (Branch).
3. Với mỗi user:
 - Dùng Upper Bound để ước lượng similarity tốt nhất có thể.
 - Nếu $\text{Upper Bound} \leq \text{ngưỡng hiện tại}$ → Cạn (cắt nhánh), bỏ qua user đó.
4. Chỉ tính cosine đầy đủ với những user có tiềm năng cao.
5. Duy trì động ngưỡng (`threshold`) bằng Min-Heap top k .

3.2.3 Chi tiết cài đặt

Đoạn mã sau mô tả phương thức lọc bỏ các ứng viên chắc chắn không nằm trong top k :

```

1 function findNeighboursWithPruning(user, normU, k):
2     // Precomputed sorted user history and suffix sum of squares
3     userSorted = sortedEntries.get(user) or empty list
4     sqSum = suffixSqSum.get(user) or [0]
5

```

```

6      // Min-heap to keep track of top-k highest similarities (for pruning)
7      topKHeap = new MinPriorityQueue()
8
9      neighbours = empty map
10
11     for each (other, otherHistory) in data:
12         if other == user:
13             continue
14
15         normV = norm.get(other)
16         if normV == 0:
17             continue
18
19         opCountSimilarity = opCountSimilarity + 1
20
21         // Get current pruning threshold (smallest similarity in top-k)
22         threshold = 0.0
23         if topKHeap.size >= k:
24             threshold = topKHeap.peek() // smallest in current top-k
25
26         // Compute similarity with early stopping (pruning)
27         sim = cosineWithEarlyStopping(userSorted, sqSum, otherHistory,
28                                     normU, normV, threshold)
29
30         if sim <= 0:
31             continue
32
33         // Add to neighbours
34         neighbours[other] = sim
35
36         // Update top-k heap for pruning future users
37         updateHeap(topKHeap, sim, k)
38
39     return neighbours
40
41 function cosineWithEarlyStopping(userSorted, sqSum, otherMap, normU, normV,
42                                 threshold):
43     denominator = normU * normV
44     partialDot = 0.0
45
46     for i = 0 to userSorted.size - 1:
47         song = userSorted[i].key
48         playCountU = userSorted[i].value.playCount
49
50         // If other user also listened to this song
51         if otherMap contains song:
52             playCountV = otherMap[song].playCount
53             partialDot += playCountU * playCountV
54
55         // Early stopping using upper bound
56         remainingMax = sqrt(sqSum[i + 1]) * normV

```

```

56     upperBound = (partialDot + remainingMax) / denominator
57
58     if upperBound <= threshold:
59         return -1                                // prune: cannot be better
60
61     // If survived all checks, return actual cosine similarity
62     return partialDot / denominator
63
64 function updateHeap(heap, sim, k):
65     heap.offer(sim)                                // insert similarity
66
67     if heap.size > k:
68         heap.poll()                                // remove the smallest similarity

```

Phân tích độ phức tạp:

- Độ phức tạp thời gian: $O(N \cdot M \cdot \log M)$ tiền xử lý một lần (sắp xếp và xây suffixSqSum); $O(N \cdot M^* + N \cdot \log k + S \cdot \log k)$ mỗi truy vấn — trong đó $M^* \leq M$ là số bước tính dot product trước khi Early Stopping kích hoạt nhờ bất đẳng thức Cauchy-Schwarz. Khi dữ liệu thưa hoặc phân phối play count lệch, $M^* \ll M$ giúp thuật toán nhanh hơn đáng kể so với Vết cạn.
- Độ phức tạp không gian: $O(N \cdot M)$ cố định cho các cấu trúc tiền xử lý (sortedEntries, suffixSqSum, norm); $O(N + S + k)$ phụ trợ mỗi truy vấn — tương đương Vết cạn nhưng đánh đổi bộ nhớ lớn hơn để đổi lấy tốc độ truy vấn.

3.3 Thuật toán Tham lam

3.3.1 Sự đánh đổi và mức độ đáp ứng hàm mục tiêu

Cần phải khẳng định rõ, việc áp dụng chiến lược tham lam vào bài toán gợi ý âm nhạc dựa trên tương tác của người dùng trong đề tài này là một phương pháp tiếp cận mang tính xấp xỉ thay vì tìm kiếm nghiệm chính xác tuyệt đối.

Xét lại hàm mục tiêu ban đầu của bài toán: Hệ thống cần tìm Top- K bài hát có tổng điểm cao nhất, trong đó điểm của mỗi bài hát được cộng dồn từ tất cả láng giềng hợp lệ trong hệ thống. Tuy nhiên, dữ liệu âm nhạc không có "cấu trúc con tối ưu" và không thỏa mãn "tính chất lựa chọn tham lam" đối với hàm mục tiêu này. Việc thuật toán tham lam ưu tiên chọn bài hát từ láng giềng giống nhất không đảm bảo bài hát đó có điểm số cao nhất toàn cục. Nguyên nhân là do một bài hát khác có thể được hàng trăm láng giềng ở mức độ tương đồng thấp hơn cùng nghe, tạo ra điểm số cộng dồn vượt trội hơn hẳn.

Do đó, thuật toán tham lam trong hệ thống thực chất không đáp ứng trọn vẹn hàm mục tiêu tối ưu hóa gốc mà chỉ áp dụng ở khâu tìm kiếm và giới hạn tập láng giềng. Việc "mượn ý tưởng" tham lam ở đây là một thử nghiệm có chủ đích, chấp nhận đánh đổi độ chính xác toàn cục và tính đa dạng của tập kết quả để đổi lấy tốc độ phản hồi. Lựa chọn tối ưu cục bộ tại mỗi bước được kỳ vọng sẽ mang lại một tập nghiệm đủ tốt để phục vụ cho các hệ thống yêu cầu xử lý thời gian thực.

3.3.2 Quy trình thực hiện tổng quát

Khác với phương pháp vét cạn phải tính toán điểm số cho toàn bộ không gian ứng viên, chiến lược tham lam bứt tốc thời gian phản hồi bằng cách liên tục đưa ra các quyết định cục bộ và ngắt luồng xử lý ngay khi đạt mục tiêu.

Quy trình bắt đầu bằng việc tính toán độ tương đồng giữa người dùng mục tiêu và phần còn lại của hệ thống, sau đó sắp xếp danh sách láng giềng theo thứ tự giảm dần. Điểm số tương đồng lúc này trở

thành tiêu chí tham lam. Tiếp theo, thuật toán ưu tiên trích xuất dữ liệu từ những láng giềng đứng đầu danh sách. Tuân thủ nghiêm ngặt nguyên tắc không quay lui, một khi bài hát hoặc láng giềng đã được đưa vào danh sách ứng viên, thuật toán sẽ không bao giờ xét lại hay so sánh với các lựa chọn phía sau. Luồng xử lý sẽ kích hoạt cơ chế ngắt sớm ngay khi thu thập đủ K bài hát, bỏ qua hoàn toàn phần dữ liệu lớn phía sau chưa được duyệt tới.

3.3.3 Thiết kế chương trình và hệ thống

Hệ thống xây dựng hai phiên bản tham lam, thể hiện hai mức độ áp dụng tính chất tham lam khác nhau trong quá trình trích xuất bài hát.

Phiên bản V1 (Greedy Neighbor):

Thuật toán giới hạn sự tham lam khi chọn lọc láng giềng. Hệ thống duyệt danh sách láng giềng từ người có độ tương đồng cao nhất trở xuống, liên tục thu thập bài hát chưa nghe vào một tập ứng viên. Ngay khi tập ứng viên này đạt đủ số lượng tối thiểu K , vòng lặp lập tức bị ngắt. Chi phí thu thập bài hát ở mỗi láng giềng chỉ phụ thuộc vào số lượng tương tác trung bình mà không tiêu tốn chi phí duyệt toàn cục. Sau khi ngắt sớm, hệ thống mới tiến hành tính điểm đầy đủ cho các bài trong tập ứng viên và dùng min-heap để lọc ra kết quả cuối cùng.

Phiên bản V2 (Greedy Song):

Phiên bản này đẩy tính chất tham lam lên mức độ cao hơn ở cả hai không gian (người dùng và bài hát). Lớp thứ nhất, thuật toán chọn láng giềng có độ tương đồng cao nhất. Lớp thứ hai, thuật toán sử dụng trực tiếp lịch sử nghe nhạc của chính láng giềng đó, tiến hành sắp xếp các bài hát theo lượt nghe giảm dần và tham lam lấy ngay những bài hát mục tiêu chưa từng nghe. Bài hát được đưa thẳng vào danh sách kết quả mà không thông qua bất kỳ bước chấm điểm nào. Vòng lặp kết thúc ngay khi tập kết quả thu thập đủ K bài.

Điểm nổi bật của cả hai phiên bản trong hệ thống là việc loại bỏ hoàn toàn các ngưỡng cắt tĩa. Thuật toán lưu lại mọi giá trị similarity kể cả khi chúng bằng 0. Tuy nhiên, nhờ cơ chế ngắt sớm tham lam, thuật toán tự nhiên miễn nhiễm với các dữ liệu nhiễu. Hệ thống luôn dừng lại trước khi chạm tới những người dùng có độ tương đồng thấp, qua đó giữ được cấu trúc lý thuyết mà không bị phụ thuộc vào các hằng số điều chỉnh.

3.3.4 Chi tiết cài đặt và đánh giá độ phức tạp

Phiên bản V1: Tham lam chọn láng giềng

Listing 6: Mã giả phiên bản Greedy V1

```

1 function RecommendV1_GreedyNeighbor(targetUser, k):
2     rankedNeighbors := {}
3
4     for each other in System.Users do
5         if other == targetUser then continue
6         sim := CosineSimilarity(targetUser, other)
7         rankedNeighbors.add((other, sim)) // Không dùng threshold
8     end for
9
10    SortDescending(rankedNeighbors, by="sim")
11
12    selectedNeighbors := {}
13    newSongPool := {}
14
15    for each (neighbor, sim) in rankedNeighbors do

```

```

16      // Kich hoạt ngat som tham lam
17      if newSongPool.size() >= k then break
18
19      selectedNeighbors[neighbor] := sim
20      for each song in neighbor.History do
21          if song not in targetUser.History then
22              newSongPool.add(song)
23          end if
24      end for
25  end for
26
27      return ScoreAndExtractTopK(targetUser, selectedNeighbors, k)
28  end function

```

Phân tích độ phức tạp (V1):

Về mặt thời gian, thao tác tính similarity mất $\mathcal{O}(|U| \cdot I_{avg})$ và sắp xếp láng giềng toàn cục tốn $\mathcal{O}(|U| \log |U|)$, với $|U|$ là tổng số người dùng và I_{avg} là số lượng bài hát trung bình mỗi người nghe. Vòng lặp gom ứng viên bị ngắt rất sớm, chỉ tốn $\mathcal{O}(C \cdot I_{avg})$ với C là số láng giềng thực tế được duyệt ($C \ll |U|$). Tổng thời gian phụ thuộc vào khâu tính similarity và chi phí dựng heap cho tập ứng viên cuối cùng: $\mathcal{O}(|U| \cdot I_{avg} + |U| \log |U| + S' \log K)$, trong đó S' là tổng số bài hát thu thập được trong pool. Về mặt không gian, thuật toán tiêu tốn $\mathcal{O}(|U|)$ để duy trì danh sách láng giềng, cộng thêm không gian phụ trợ $\mathcal{O}(S')$ để lưu trữ tập bài hát ứng viên trước khi đưa vào Heap.

Phiên bản V2: Tham lam chọn bài hát

Listing 7: Mã giả phiên bản Greedy V2

```

1  function RecommendV2_GreedySong(targetUser, k):
2      // ... (Cac bước tính toán và sắp xếp láng giềng tương tự V1)
3      SortDescending(rankedNeighbors, by="sim")
4
5      results := {}
6      seenSongs := targetUser.History
7
8      for each (neighbor, sim) in rankedNeighbors do
9          if results.size() >= k then break // Ngat som vong lap lang gieng
10
11         // Tiêu chí tham lam 2: Sắp xếp bài hát của từng láng giềng
12         neighborSongs := SortDescending(neighbor.History, by="playCount")
13
14         for each song in neighborSongs do
15             if results.size() >= k then break // Ngat som vong lap bai hat
16
17             if song not in seenSongs then
18                 results.add(song)
19                 seenSongs.add(song)
20             end if
21         end for
22     end for
23
24     return results
25 end function

```

Phân tích độ phức tạp (V2):

Về mặt thời gian, bước đầu vẫn tốn $\mathcal{O}(|U| \cdot I_{avg} + |U| \log |U|)$. Khác biệt nằm ở bước trích xuất: thuật toán tiến hành sắp xếp bài hát của từng lảng giềng được xét, tiêu tốn $\mathcal{O}(I_{avg} \log I_{avg})$ cho mỗi người. Do quá trình ngắt sớm diễn ra cực nhanh, số lượng lảng giềng C phải duyệt thường rất nhỏ. Tổng thời gian xử lý là $\mathcal{O}(|U| \cdot I_{avg} + |U| \log |U| + C \cdot I_{avg} \log I_{avg})$.

Mặc dù V2 triệt tiêu hoàn toàn vòng lặp cộng dồn điểm số phức tạp, cả hai phiên bản vẫn tồn tại chi phí $\mathcal{O}(|U| \log |U|)$ ở bước sắp xếp người dùng ban đầu. Điều này cho thấy thuật toán chưa thực sự "tham lam" toàn phần từ giai đoạn gốc. Khi không gian người dùng $|U|$ tăng lên mức khổng lồ, chi phí sắp xếp sẽ trở thành điểm nghẽn kìm hãm tốc độ của hệ thống. Về mặt không gian, ngoài chi phí $\mathcal{O}(|U|)$ cho danh sách lảng giềng và $\mathcal{O}(K)$ cho tập kết quả, thuật toán cần bộ nhớ để duy trì cấu trúc băm seenSongs phục vụ tra cứu $\mathcal{O}(1)$. Tổng không gian lưu trữ phụ là $\mathcal{O}(|U| + S_{target} + K)$.

3.4 Thuật toán caching kết hợp tiền xử lý

3.4.1 Sự đánh đổi và mức độ áp dụng lý thuyết quy hoạch động

Kỹ thuật Caching được triển khai trong đề tài này vay mượn ý tưởng của quy hoạch động về mặt lưu trữ và tái sử dụng, nhưng đây không phải là một thuật toán DP thuần túy. Xét về mặt toán học, bài toán tính độ tương đồng giữa các người dùng hoàn toàn là các phép tính phẳng, độc lập và không sở hữu "cấu trúc con tối ưu". Việc giải quyết xong độ tương đồng của cặp người dùng (A, B) không cung cấp công thức truy hồi nào để giải quyết cặp (A, C) . Do đó, phương pháp này được định nghĩa chính xác hơn là kỹ thuật lưu trữ đệm kết hợp tiền xử lý cơ sở dữ liệu.

Mục tiêu của thuật toán này là giảm thời gian tìm kiếm các lảng giềng ở giai đoạn đầu (pha 1). Tuy nhiên, kết quả thực nghiệm cho thấy với những bộ dữ liệu thừa, việc sử dụng các cấu trúc dữ liệu lưu tạm có thể làm phát sinh thêm chi phí xử lý. Cụ thể, bảng băm cần thêm thời gian cho việc tính toán hàm băm, quản lý con trỏ và giải phóng bộ nhớ, nên đôi khi kém hiệu quả hơn so với việc sử dụng mảng thông thường. Ngoài ra, sau khi xác định được các lảng giềng phù hợp, hệ thống vẫn phải thực hiện pha 2 để tính điểm. Quá trình này yêu cầu duyệt qua các cấu trúc dữ liệu đã tạo trước đó, dẫn đến nhiều lần truy cập bộ nhớ không liên tục và làm tăng chi phí xử lý. Mặc dù vậy, nhóm vẫn lựa chọn cách tiếp cận này vì nó giúp loại bỏ các vòng lặp không cần thiết và cho phép tái sử dụng các trạng thái hoặc kết quả tính toán đã có, từ đó phản ánh tốt hơn cách hoạt động của các hệ thống tối ưu trong thực tế.

3.4.2 Quy trình thực hiện tổng quát

Khác với phương pháp vét cạn phải duyệt qua tất cả người dùng và tính toán lại độ tương đồng từ đầu mỗi khi có truy vấn, phương pháp này cải thiện hiệu năng bằng cách lưu trữ và tái sử dụng các kết quả trung gian đã được tính toán trước. Ở phiên bản Top-Down, độ tương đồng giữa hai người dùng sau khi được tính toán sẽ được lưu vào bộ nhớ đệm. Khi gặp lại cùng một cặp người dùng ở các truy vấn sau, hệ thống chỉ cần lấy kết quả từ bộ nhớ thay vì tính toán lại. Ở phiên bản Bottom-Up, hệ thống thực hiện bước tiền xử lý dữ liệu trước khi nhận truy vấn. Cụ thể, một chỉ mục ngược được xây dựng để lưu danh sách người dùng đã nghe từng bài hát, đồng thời độ dài vector của mỗi người dùng cũng được tính sẵn và lưu lại. Nhờ đó, khi có truy vấn, hệ thống chỉ cần xét những người dùng có bài hát chung với người dùng mục tiêu thay vì duyệt toàn bộ tập dữ liệu. Sau khi tính được độ tương đồng, chỉ những người dùng có giá trị tương đồng lớn hơn 0 mới được giữ lại làm lảng giềng. Bước lọc này giúp giảm số lượng phép tính không cần thiết trong giai đoạn chấm điểm. Cuối cùng, hệ thống tính điểm cho các bài hát ứng viên dựa trên thông tin từ các lảng giềng đã tìm được và sử dụng cấu trúc min-heap để duy trì danh sách Top-K bài hát có điểm số cao nhất trước khi trả kết quả cho người dùng.

3.4.3 Thiết kế chương trình và hệ thống

Dựa trên hai hướng tối ưu hóa đã trình bày, caching được triển khai dưới hai phiên bản khác nhau.

Phiên bản V1 (Top-Down Memoization):

Bộ nhớ đệm được thiết kế dưới dạng Hash Map lưu trữ theo cặp khóa-giá trị. Khóa lưu trữ là chuỗi định danh của hai người dùng được phân tách bởi ký tự `||` và sắp xếp theo thứ tự từ điển nhằm đảm bảo tính đối xứng ($\text{sim}(A, B) = \text{sim}(B, A)$). Quá trình diễn ra theo cơ chế đánh giá lười. Chỉ khi một truy vấn thực tế xuất hiện và hệ thống ghi nhận Cache Miss, nó mới gọi hàm tính toán và điền dữ liệu vào bộ nhớ đệm phục vụ cho tương lai.

Phiên bản V2 (Bottom-Up Precomputation kết hợp Inverted Index):

Hệ thống được thiết kế với cơ chế khởi động trước. Thay vì cố gắng tính trước độ tương đồng của ma trận $\mathcal{O}(|U|^2)$ rất tốn kém, thuật toán chỉ xử lý các thành phần cần thiết của công thức similarity. Hệ thống sử dụng vòng lặp duyệt qua lịch sử tương tác một lần duy nhất để: (1) Xây dựng cấu trúc chỉ mục ngược ánh xạ từ bài hát sang danh sách người dùng và lượt nghe; (2) Tính toán sẵn độ dài vector của toàn bộ người dùng. Khi truy vấn diễn ra, hệ thống chỉ việc duyệt qua các bài hát người dùng mục tiêu đã nghe, sử dụng chỉ mục ngược để tìm ra tập người dùng có vùng giao, cộng dồn tích vô hướng và chia cho các norm đã tính sẵn.

3.4.4 Chi tiết cài đặt và đánh giá độ phức tạp

Phiên bản V1: Lưu đệm từ trên xuống (Top-Down Memoization)

Listing 8: Mã giả phiên bản Caching V1

```

1 function RecommendV1_TopDown(targetUser, k):
2     neighbors := {}
3     for each otherUser in System.Users do
4         if otherUser == targetUser then continue
5
6         cacheKey := getMemoKey(targetUser, otherUser)
7
8         if cacheKey exists in memoCache then
9             sim := memoCache[cacheKey] // Cache Hit
10        else
11            sim := CosineSimilarity(targetUser, otherUser) // Cache Miss
12            memoCache[cacheKey] := sim
13        end if
14
15        if sim > 0 then
16            neighbors[otherUser] := sim
17        end if
18    end for
19    return ScoreAndExtractTopK(targetUser, neighbors, k)
20 end function

```

Phân tích độ phức tạp (V1):

Về mặt thời gian, trong trường hợp xấu nhất (bộ nhớ đệm trống), hệ thống tính độ tương đồng cho tất cả người dùng khác, tốn $\mathcal{O}(|U| \cdot I_{avg})$. Trong trường hợp tốt nhất, khi toàn bộ giá trị tương đồng đã tồn tại trong bộ nhớ đệm, hệ thống chỉ cần duyệt qua tập người dùng và thực hiện các phép tra cứu Hash Map với chi phí trung bình $\mathcal{O}(1)$. Khi đó, chi phí tìm láng giềng giảm xuống $\mathcal{O}(|U|)$. Tổng thời gian truy vấn bao gồm bước chấm điểm và trích xuất Top-K xấp xỉ $\mathcal{O}(|U| + S_{cand} \log K)$, với S_{cand} là tổng số bài

hát ứng viên được đưa vào tính điểm.

Về mặt không gian, bộ nhớ đệm phát triển theo thời gian thực. Do sử dụng chuỗi khóa (string key), thuật toán tiêu tốn thêm bộ nhớ cấp phát chuỗi. Kích thước của cấu trúc Hash Map tiệm cận $\mathcal{O}(|E_{users}|)$, với $|E_{users}|$ là số cặp người dùng đã từng được truy vấn. Trong trường hợp cực đại, khi mọi cặp người dùng đều đã được truy vấn ít nhất một lần, số phần tử lưu trữ trong bộ nhớ đệm có thể đạt $\mathcal{O}(|U|^2)$, thể hiện rõ sự đánh đổi không gian bộ nhớ để tối ưu hóa tốc độ xử lý.

Phiên bản V2: Tiền xử lý từ dưới lên (Bottom-Up Precomputation)

Listing 9: Mã giả phiên bản Caching V2 (Inverted Index + Vector Norms)

```

1 function BuildDPTable():
2     if dpTableBuilt then return
3
4     // 1. Duyệt 1 lần để xây dựng Chi mục ngược và tính Norm
5     for each (user, userHistory) in System.Data do
6         normSq := 0.0
7         for each (song, playCount) in userHistory do
8             dpSongTable[song][user] := playCount
9             normSq := normSq + (playCount * playCount)
10        end for
11        userNorms[user] := sqrt(normSq)
12    end for
13    dpTableBuilt := true
14 end function
15
16 function RecommendV2_BottomUp(targetUser, k):
17     BuildDPTable()
18     targetHistory := System.Data[targetUser]
19     normTarget := userNorms[targetUser]
20     if normTarget == 0.0 then return {}
21
22     dotProducts := {}
23     // 2. Giống hàng và chỉ tính tích vô hướng cho user có chung bài hát
24     for each (song, playTarget) in targetHistory do
25         listeners := dpSongTable[song]
26         for each (v, listenerPlay) in listeners do
27             if v == targetUser then continue
28             dotProducts[v] += playTarget * listenerPlay
29         end for
30     end for
31
32     // 3. Áp dụng norm để ra Cosine cuối cùng
33     neighbors := {}
34     for each (v, dot) in dotProducts do
35         normV := userNorms[v]
36         if normV > 0.0 then
37             sim := dot / (normTarget * normV)
38             if sim > 0 then neighbors[v] := sim
39         end if
40     end for

```

```

41
42     return ScoreAndExtractTopK(targetUser, neighbors, k)
43 end function

```

Phân tích độ phức tạp (V2):

Về mặt thời gian, chi phí tiền xử lý là $\mathcal{O}(N_{interactions})$, do hệ thống chỉ cần duyệt qua toàn bộ dữ liệu tương tác một lần duy nhất để xây dựng chỉ mục ngược và tính trước độ dài vector của từng người dùng. Cách tiếp cận này tránh được việc phải tính toán và lưu trữ trước toàn bộ ma trận độ tương đồng người dùng - người dùng với chi phí tiềm năng $\mathcal{O}(|U|^2)$. Khi truy vấn trực tuyến, chi phí tính toán gồm hai thành phần: quá trình cộng dồn tích vô hướng tốn $\mathcal{O}(|I_{target}| \cdot L_{avg})$, và quá trình chuẩn hóa cosine cho tập ứng viên tốn $\mathcal{O}(|N_{cand}|)$, trong đó $|I_{target}|$ là số bài hát người dùng mục tiêu đã nghe, L_{avg} là số người nghe trung bình của một bài hát, và $|N_{cand}|$ là số láng giềng có ít nhất một bài hát chung.

Về mặt không gian, cấu trúc chỉ mục ngược dpSongTable yêu cầu $\mathcal{O}(N_{interactions})$ bộ nhớ, trong khi mảng userNorms yêu cầu $\mathcal{O}(|U|)$. Nhờ chỉ lưu trữ các tương tác thực sự tồn tại, phương pháp này tránh được chi phí lưu trữ bậc hai $\mathcal{O}(|U|^2)$ của ma trận độ tương đồng đầy đủ, qua đó giảm đáng kể nhu cầu tài nguyên bộ nhớ so với các phương pháp vét cạn ma trận dày đặc.

3.5 Thuật toán Heuristic

3.5.1 Quy trình thực hiện

Thuật toán Heuristic được triển khai theo hai phiên bản với cùng ý tưởng lọc ứng viên, nhưng khác nhau về cách xác định overlap:

- **V1 — FilterOverlap:** Duyệt tuần tự tập bài hát của từng người dùng khác, đếm số bài chung với người dùng u , dừng sớm (early stop) khi đã đủ ngưỡng.
- **V2 — InvertedIndex:** Xây dựng một *inverted index* ánh xạ từ mỗi bài hát sang tập người dùng đã nghe bài đó. Khi tính overlap cho u , chỉ cần duyệt các bài hát trong S_u và tra bảng ngược, thu được tất cả ứng viên và đếm overlap trong $\mathcal{O}(|S_u| \times \bar{d})$ thay vì $\mathcal{O}(U \times |S_v|)$.

Sau bước lọc, pipeline tiếp tục giống các thuật toán khác: tính cosine similarity trên tập ứng viên đã lọc, tính điểm bài hát, rút ra Top-K bằng min-heap.

3.5.2 Thiết kế chương trình và hệ thống

Dựa trên cùng một tư tưởng cốt lõi là giảm số lượng phép tính similarity không cần thiết thông qua cơ chế lọc ứng viên, hệ thống Heuristic được triển khai thành hai phiên bản với hai chiến lược xác định overlap khác nhau:

- Phiên bản V1 - recommendV1_FilterOverlap: Hệ thống áp dụng heuristic trực tiếp trên dữ liệu gốc. Với mỗi người dùng mục tiêu u , thuật toán lần lượt duyệt qua toàn bộ người dùng còn lại và kiểm tra mức độ giao nhau giữa hai tập bài hát. Quá trình kiểm tra được tối ưu bằng cơ chế early stopping. Ngay khi số lượng bài hát chung đạt tới ngưỡng yêu cầu, người dùng đó lập tức được đưa vào tập ứng viên mà không cần tiếp tục duyệt phần dữ liệu còn lại.

Cách tiếp cận này không yêu cầu thêm cấu trúc dữ liệu phụ trợ nên giữ được mức sử dụng bộ nhớ thấp. Tuy nhiên, hệ thống vẫn phải thực hiện nhiều phép duyệt tuyến tính trên tập dữ liệu người dùng, khiến hiệu năng còn phụ thuộc đáng kể vào kích thước dữ liệu đầu vào.

- Phiên bản V2 - recommendV2_InvertedIndex: Phiên bản này thay đổi hoàn toàn hướng tiếp cận bằng cách tổ chức lại dữ liệu dưới dạng inverted index. Mỗi bài hát được ánh xạ tới danh sách những người dùng đã từng nghe bài hát đó.

Khi xử lý truy vấn cho người dùng u , hệ thống không còn cần duyệt qua toàn bộ tập người dùng nữa. Thay vào đó, thuật toán chỉ duyệt các bài hát thuộc S_u , sau đó tra cứu bảng ngược để thu được các ứng viên có giao nhau thực sự với u .

Trong quá trình này, mỗi lần một người dùng xuất hiện thông qua inverted index, bộ đếm overlap của người đó được tăng thêm một đơn vị. Sau khi hoàn tất, thuật toán chỉ giữ lại những người dùng đạt ngưỡng overlap yêu cầu.

Thiết kế này chuyển trọng tâm chi phí tính toán từ việc quét toàn bộ không gian người dùng sang việc khai thác trực tiếp các điểm giao thực tế trong dữ liệu. Đối với các tập dữ liệu sparse, số lượng người dùng xuất hiện trong inverted index thường nhỏ hơn rất nhiều so với tổng số người dùng toàn hệ thống, giúp giảm mạnh chi phí xử lý.

Khác với các phương pháp tính similarity toàn phần, hệ thống Heuristic không cố gắng tối ưu công thức cosine similarity mà tập trung vào việc giảm kích thước tập ứng viên trước khi bước vào giai đoạn tính toán chính thức. Điều này giúp thuật toán vẫn giữ nguyên độ chính xác của phép đo similarity trên các ứng viên được chọn, đồng thời loại bỏ phần lớn các phép tính dư thừa.

Quá trình xử lý của hệ thống được tổ chức thành ba giai đoạn chính:

1. Giai đoạn lọc ứng viên: Hệ thống áp dụng heuristic overlap để xác định những người dùng có khả năng thực sự tương đồng với người dùng mục tiêu. Đây là bước quyết định trực tiếp hiệu năng tổng thể của thuật toán vì nó kiểm soát số lượng phép tính similarity phát sinh về sau.
2. Giai đoạn tính similarity: Sau khi tập ứng viên được rút gọn, hệ thống mới tiến hành tính cosine similarity. Việc tính toán lúc này chỉ diễn ra trên một phần nhỏ dữ liệu thay vì toàn bộ ma trận người dùng $\times$ người dùng như phương pháp vét cạn.
3. Giai đoạn sinh gợi ý: Từ danh sách láng giềng đã được lọc, thuật toán tính weighted score cho các bài hát chưa nghe, sau đó sử dụng Min-Heap kích thước cố định K để duy trì Top-K bài hát có điểm số cao nhất.

Bản chất của heuristic trong hệ thống: Thuật toán được xây dựng dựa trên giả định thực nghiệm rằng các cặp người dùng có overlap quá thấp gần như không đóng góp meaningful similarity cho hệ gợi ý. Vì vậy, thay vì dành tài nguyên để tính cosine cho mọi cặp người dùng, hệ thống chủ động cắt tỉa không gian tìm kiếm ngay từ đầu bằng cơ chế threshold.

Khác với các kỹ thuật approximation thay đổi trực tiếp công thức similarity, heuristic ở đây chỉ tác động lên bước chọn ứng viên. Điều này giúp thuật toán vẫn giữ nguyên công thức cosine similarity chuẩn trên tập dữ liệu còn lại, đồng thời đạt được mức giảm chi phí xử lý đáng kể trên dữ liệu sparse thực tế.

Tuy nhiên, sự đánh đổi của chiến lược này là khả năng bỏ sót một số láng giềng tiềm năng nếu overlap chưa đạt ngưỡng yêu cầu. Giá trị α vì vậy trở thành tham số điều khiển trực tiếp mức độ đánh đổi giữa tốc độ xử lý và độ bao phủ kết quả của hệ thống.

3.5.3 Chi tiết cài đặt

Đoạn mã sau mô tả phương thức lọc ứng viên theo inverted index (V2):

Listing 10: Lọc ứng viên bằng Inverted Index (HeuristicRecommender V2)

```

1 function filterDataBySong(user):
2     // Step 1: Count overlapping songs with other users using inverted index
3     overlapCount = empty map // User -> number of shared songs
4
5     for each song in data[user].keys:
```

```

6         listeners = invertedIndex.get(song) or empty set
7
8         for each other in listeners:
9             if other != user:
10                 overlapCount[other] = overlapCount[other] + 1    // increment
11
12     // Step 2: Build filtered dataset
13     filteredData = empty map
14
15     // Always include the target user
16     filteredData[user] = data[user]
17
18     userSongCount = size of data[user]
19     alpha = 0.2                                // overlap threshold = 20%
20
21     // Step 3: Keep only users with sufficient song overlap
22     for each (other, sharedCount) in overlapCount:
23         otherSongCount = size of data[other]
24
25         // Minimum required overlapping songs
26         threshold = alpha * minimum(userSongCount, otherSongCount)
27
28         if sharedCount >= threshold:
29             filteredData[other] = data[other]
30
31     return filteredData

```

Phân tích độ phức tạp:

- **V1 — FilterOverlap:** $O(U \times \bar{d})$ với $\bar{d}$ là số bài hát trung bình mỗi người dùng. Duyệt toàn bộ U người dùng, nhưng dừng sớm khi đủ ngưỡng.
- **V2 — InvertedIndex:** $O(|S_u| \times \bar{d}_{song})$ cho bước lọc, với $\bar{d}_{song}$ là số người dùng trung bình mỗi bài hát. Tổng thời gian lọc giảm từ $O(U \times \bar{d})$ xuống $O(|S_u| \times \bar{d}_{song}) \ll O(U \times \bar{d})$ khi dữ liệu thưa.
- **Không gian:** V2 cần thêm $O(U \times \bar{d})$ cho inverted index, đây là đánh đổi bộ nhớ lấy tốc độ.

4 KẾT QUẢ THỰC NGHIỆM

4.1 Môi trường thực nghiệm

Để đảm bảo tính khách quan và chính xác khi đối chiếu hiệu năng giữa các thuật toán, các thực nghiệm được tiến hành trên một môi trường phần cứng và phần mềm đồng nhất.

1. Cấu hình phần cứng và phần mềm

- CPU: AMD Ryzen 5 7520U with Radeon Graphics (2.80 GHz)
- RAM: 8.00 GB
- Hệ điều hành: Windows 11
- Môi trường thực thi: JDK 21 (version 21.0.9)

2. Phương pháp đo lường và đánh giá

Để đảm bảo kết quả thực nghiệm phản ánh đúng hiệu năng của thuật toán, quá trình đo lường được thực hiện theo một số nguyên tắc nhằm giảm ảnh hưởng từ môi trường thực thi Java. Trước khi tiến hành đo chính thức, thuật toán được chạy thử trên 10 người dùng đầu tiên và lặp lại 2 lần. Mục đích là để bộ biên dịch Just-In-Time (JIT) của Java hoàn tất việc tối ưu mã nguồn và nạp các lớp cần thiết vào bộ nhớ. Nhờ đó, các lần đo sau phản ánh chính xác hơn hiệu năng thực tế của thuật toán.

Thời gian thực thi được ghi nhận bằng hàm `System.nanoTime()` với độ chính xác ở mức nano giây. Ngoài thời gian chạy, hệ thống còn theo dõi số lượng các phép toán quan trọng như số lần tính độ tương đồng Cosine (`opCountSimilarity`), số thao tác trên cấu trúc Heap (`opCountHeap`) và số lần thực hiện bước chấm điểm (`opCountScore`). Các chỉ số này giúp đánh giá hiệu quả thuật toán một cách chi tiết hơn so với việc chỉ dựa vào thời gian thực thi.

Trước và sau mỗi lần thực nghiệm, hệ thống chủ động gọi cơ chế thu gom rác (`System.gc()`) để hạn chế ảnh hưởng của các đối tượng không còn được sử dụng. Lượng bộ nhớ tiêu thụ được xác định thông qua chênh lệch giữa tổng bộ nhớ heap đang sử dụng trước và sau khi chạy thuật toán. Ngoài ra, phần bộ nhớ phát sinh từ các cấu trúc dữ liệu phụ trợ cũng được ước lượng và cộng vào kết quả đo để phản ánh đầy đủ chi phí bộ nhớ của từng phương pháp.

4.2 Bộ dữ liệu và kịch bản thử nghiệm

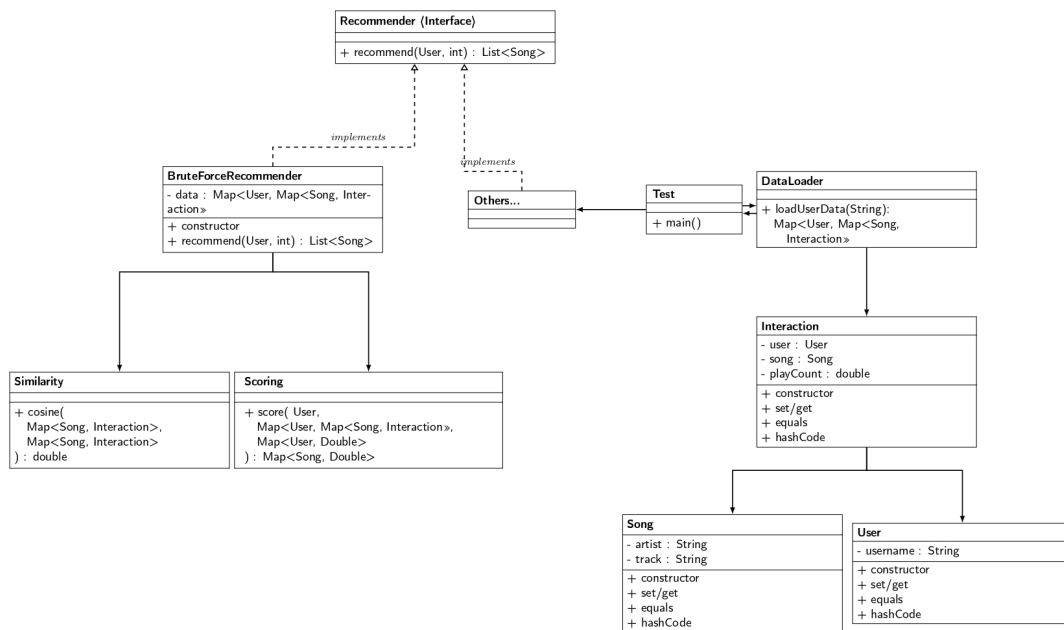
1. Bộ dữ liệu

Đề tài sử dụng bộ dữ liệu trích xuất từ Last.fm 1K Dataset. Dữ liệu thô sau khi được tiền xử lý đã được tổng hợp thành một tệp danh sách các tương tác gồm 500 users.

- Đặc trưng dữ liệu: Ma trận thưa gồm các bản ghi thể hiện tần suất nghe nhạc của mỗi người dùng đối với các bài hát.
- Khối lượng thử nghiệm: Toàn bộ tệp chứa dữ liệu của đúng 500 người dùng. Sau khi nạp thông qua lớp `DataLoader`, danh sách này được xáo trộn ngẫu nhiên với seed cố định để đảm bảo tính tái lập giữa các lần chạy.

2. Kiến trúc tổ chức luồng kiểm thử

Để quản lý việc nạp dữ liệu và chuyển giao qua lại giữa các phiên bản thuật toán, hệ thống được thiết kế tuân thủ các nguyên tắc hướng đối tượng. Sự liên kết giữa luồng dữ liệu và các bộ gọi ý được thể hiện qua sơ đồ lớp dưới đây.



Hình 1: Sơ đồ lớp kiến trúc của hệ thống gợi ý âm nhạc

Như thể hiện trong hình 1, lớp **Test** đóng vai trò điều phối trung tâm. Nó gọi **DataLoader** để khởi tạo các thực thể **User**, **Song**, **Interaction**, sau đó truyền ma trận dữ liệu này vào lớp trừu tượng **Recommender** để tiến hành các kịch bản đánh giá trên các lớp thuật toán con.

3. Các kịch bản thử nghiệm

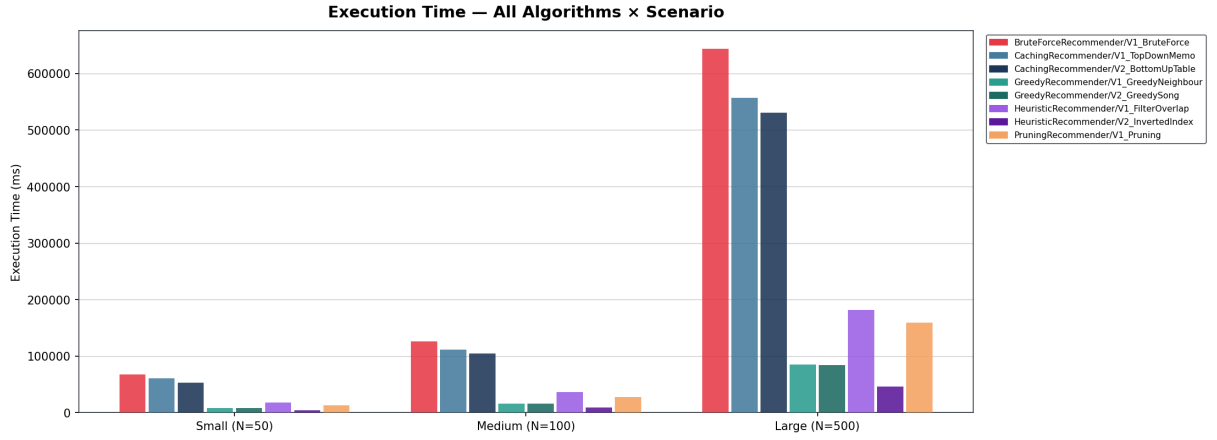
Bài toán yêu cầu gợi ý Top- K bài hát (với thiết lập $K = 5$) cho mỗi người dùng. Để đánh giá mức độ tăng trưởng về chi phí theo kích thước dữ liệu, hệ thống chia thành 3 kịch bản kiểm thử tăng dần với các tập con của toàn bộ 500 người dùng:

- Kịch bản Small ($N = 50$): Kiểm tra tính đúng đắn và tốc độ xử lý trên tập dữ liệu nhỏ. Tại mức này, sự khác biệt giữa các thuật toán thường chưa rõ rệt.
- Kịch bản Medium ($N = 100$): Đánh giá mức độ ổn định.
- Kịch bản Large ($N = 500$): Chạy trên toàn bộ dữ liệu. Đây là kịch bản quan trọng nhất để chứng minh tính vượt trội các thuật toán trong việc cắt giảm số vòng lặp và tối ưu hóa bộ nhớ đệm.

Tất cả các thuật toán đều được yêu cầu sinh ra kết quả cho 3 kịch bản này. Một thuật toán được đánh giá là đạt (PASS) nếu nó sinh ra được danh sách gợi ý hợp lệ cho ít nhất 50% số lượng người dùng trong tập kiểm thử đó. Tuy nhiên do mỗi thuật toán có thể có hoặc không ngưỡng cắt cụ thể nên trạng thái PASS ở đây không có nghĩa là thuật toán đã cho kết quả tốt nhất.

4.3 Kết quả thực nghiệm

4.3.1 Thời gian thực thi



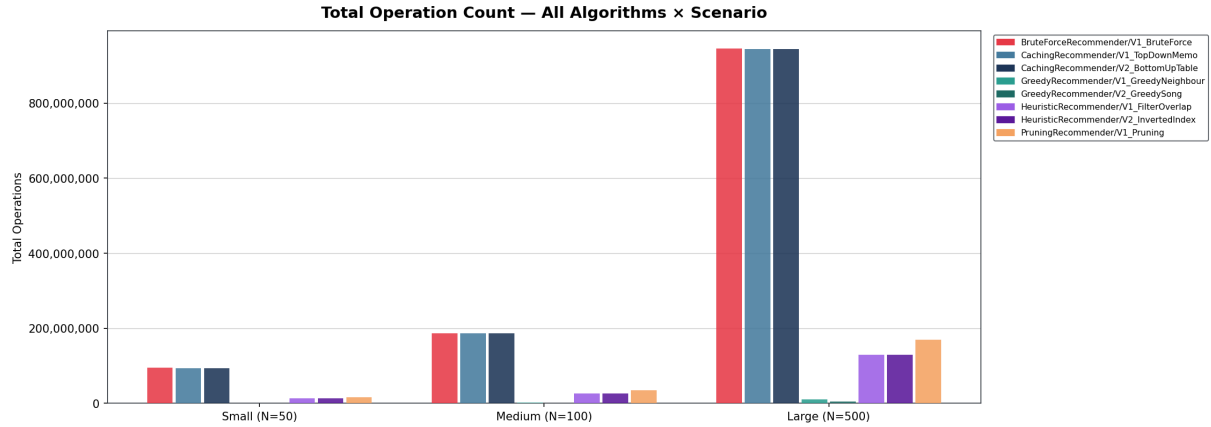
Hình 2: Thời gian thực thi của các thuật toán theo kích bản (ms)

Biểu đồ Hình 2 biểu diễn thời gian thực thi tổng cộng trên toàn bộ người dùng của từng kịch bản cho tám biến thể thuật toán ở ba mức Small, Medium và Large. Ngay từ kịch bản Small (N=50), sự phân hóa giữa các nhóm đã bắt đầu rõ ràng. Thời gian của BruteForce khoảng 65.000 ms, trong khi GreedyV1_GreedyNeighbour và GreedyV2_GreedySong chỉ cần lần lượt ≈ 9.000 ms và ≈ 8.000 ms. Đặc biệt, HeuristicV2_InvertedIndex xử lý xong kịch bản chỉ trong khoảng 3.000 ms. Hai phiên bản Caching ngược lại có thời gian ≈ 60.000 ms (V1_TopDownMemo) và ≈ 53.000 ms (V2_BottomUpTable) - gần sát với BruteForce. Đây là kết quả trái ngược với kỳ vọng ban đầu về lợi ích của lưu đệm.

Ở kịch bản Medium (N=100), BruteForce tăng lên ≈ 128.000 ms, phản ánh xu hướng tăng trưởng gần bậc hai. Caching V1 (≈ 111.000 ms) và V2 (≈ 105.000 ms) vẫn không tạo ra khoảng cách đáng kể so với BruteForce. Các thuật toán Greedy tiếp tục duy trì mức thấp ở ≈ 15.000 ms cho cả hai phiên bản, trong khi HeuristicV2 chỉ cần khoảng 8.000 ms.

Đến kịch bản Large (N=500), tương quan lực lượng giữa các nhóm trở nên cực kỳ rõ ràng. BruteForce cần tới ≈ 643.000 ms. CachingV1_TopDownMemo tiêu tốn ≈ 557.000 ms và CachingV2_BottomUpTable là ≈ 530.000 ms - tức là cả hai vẫn nằm trong cùng bậc độ lớn với BruteForce dù được kỳ vọng là nhanh hơn đáng kể. Ở phía ngược lại, GreedyV1_GreedyNeighbour hoàn thành ở ≈ 85.600 ms và GreedyV2_GreedySong ở ≈ 84.400 ms - nhanh hơn BruteForce hơn bảy lần. HeuristicV2_InvertedIndex là thuật toán nhanh nhất với ≈ 46.200 ms, trong khi PruningV1 (≈ 159.600 ms) và HeuristicV1_FilterOverlap (≈ 181.200 ms) nằm ở tầm trung.

4.3.2 Tổng số phép tính



Hình 3: Tổng số phép tính của các thuật toán theo kịch bản

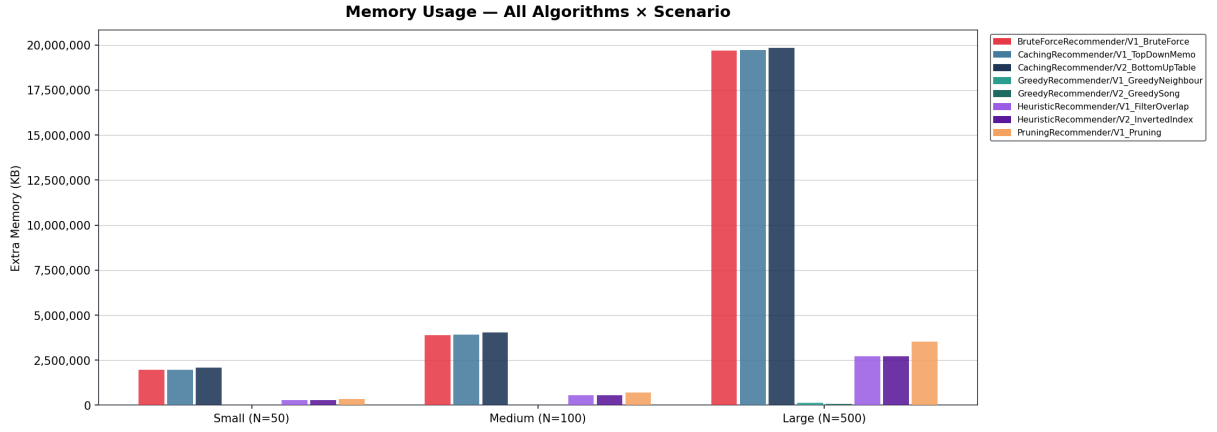
Biểu đồ Hình 3 phản ánh số lượng phép toán thực sự được thực hiện - một thước đo khách quan không bị ảnh hưởng bởi đặc thù của môi trường thực thi. Kết quả ở đây làm sáng tỏ một nghịch lý quan trọng liên quan đến Caching và giải thích tại sao nhóm thuật toán này không đạt được tăng tốc đáng kể.

Tại kịch bản Large (N=500), BruteForce thực hiện khoảng 940×10^6 phép tính. Cả CachingV1_TopDownMemo lẫn CachingV2_BottomUpTable cũng đạt mức tổng phép tính tương đương. Dù cơ chế lưu đệm tránh được việc tính lại cosine similarity cho một số cặp người dùng, phần chi phí áp đảo lại nằm ở thao tác Heap và Scoring - và hai thao tác này vẫn phải thực hiện đầy đủ với toàn bộ không gian bài hát ở mỗi truy vấn, khiến tổng phép tính gần như không giảm so với BruteForce.

Các thuật toán Tham lam (GreedyV1 và GreedyV2) có tổng phép tính ở mức gần như không đáng kể trên biểu đồ - chỉ vài triệu phép so với gần một tỉ của BruteForce. Đây là hệ quả trực tiếp của cơ chế ngắt sớm triệt để: khi đã tích lũy đủ K bài hát ứng viên từ những láng giềng tương đồng nhất, thuật toán dừng xử lý hoàn toàn mà không duyệt thêm bất kỳ người dùng hay bài hát nào. PruningV1 đạt $\approx 165 \times 10^6$ phép tính nhờ dừng sớm tính toán cosine khi kết quả trung gian đã cho thấy ứng viên hiện tại không thể lọt vào Top-K. Hai phiên bản Heuristic dừng ở mức $\approx 130 \times 10^6$ phép tính, cho thấy bộ lọc ứng viên đồng thời thu hẹp cả ba loại chi phí: Similarity, Score và Heap.

Xét theo chiều tăng trưởng khi N tăng từ 50 lên 500, BruteForce và Caching tăng khoảng $9,4\times$ - xác nhận tốc độ bậc hai. Heuristic và Pruning tăng chậm hơn đáng kể, trong khi Greedy gần như không thay đổi bậc độ lớn, khẳng định lợi thế ngày càng rõ ràng của các phương pháp lọc ứng viên khi dữ liệu mở rộng.

4.3.3 Sử dụng bộ nhớ



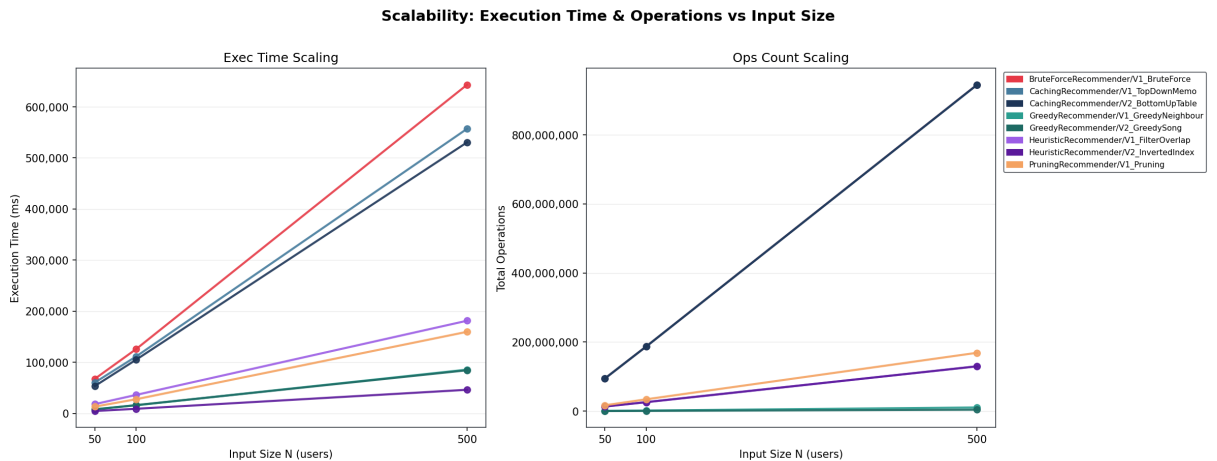
Hình 4: Bộ nhớ phụ sử dụng của các thuật toán theo kích bản (KB)

Hình 4 đo lường bộ nhớ phụ mà mỗi thuật toán cần ngoài dữ liệu gốc, là chiều đánh giá quan trọng bổ sung cho tốc độ khi lựa chọn giải pháp triển khai thực tế.

Tại kích bản Large (N=500), BruteForce, CachingV1 và CachingV2 đều tiêu thụ bộ nhớ phụ ở mức xấp xỉ nhau, vào khoảng 20.000.000 KB. Sự tương đồng này là do cả ba phương pháp đều phải duy trì các cấu trúc lưu trữ lớn để phục vụ duyệt toàn bộ không gian người dùng và bài hát ở mỗi truy vấn. Caching không chỉ thất bại trong việc giảm phép tính mà còn không giảm được gánh nặng bộ nhớ - chi phí lưu đệm chỉ thêm vào mà không bù đắp được ở khâu nào khác.

Ngược lại, GreedyV1 và GreedyV2 tiêu thụ bộ nhớ phụ gần như không đáng kể - hai cột trên biểu đồ hầu như không nhìn thấy dù trục dọc ở thang 20×10^6 KB. Cơ chế ngắt sớm giúp thuật toán không cần duy trì bảng người dùng hay bảng điểm đầy đủ, chỉ cần lưu tạm thời một pool bài hát nhỏ trong quá trình xử lý. HeuristicV1 và V2 ở mức $\approx 2.500.000$ KB - tiết kiệm hơn BruteForce tám lần - nhờ chỉ lưu similarity map của tập ứng viên đã được lọc. PruningV1 tiêu tốn thêm một chút ở mức $\approx 3.500.000$ KB do phải duy trì thêm cấu trúc phục vụ quá trình cắt tĩa và min-heap Top-K liên tục. Nhìn chung, bộ nhớ sử dụng có mối tương quan chặt chẽ với số lượng ứng viên mà thuật toán xử lý: cắt giảm sớm không gian ứng viên không chỉ tiết kiệm thời gian mà còn giảm đáng kể áp lực bộ nhớ.

4.3.4 Khả năng mở rộng



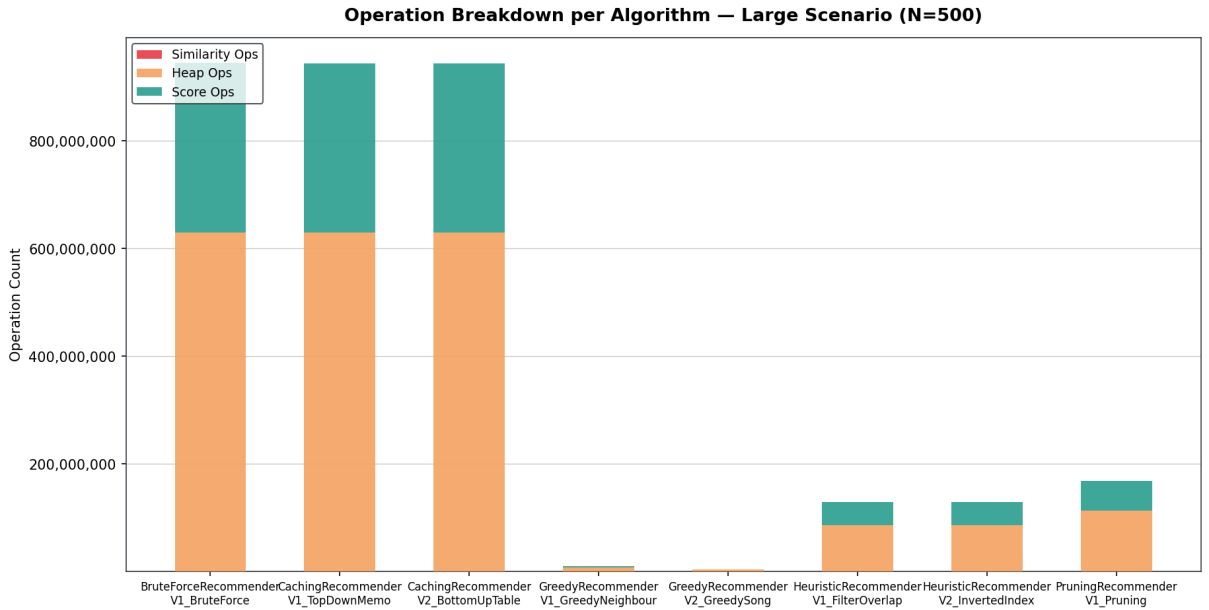
Hình 5: Khả năng mở rộng: thời gian và số phép tính theo kích thước đầu vào N

Hình 5 thể hiện tốc độ tăng trưởng của thời gian thực thi (đồ thị trái) và tổng phép tính (đồ thị phải) khi N tăng từ 50 lên 100 rồi 500, qua đó xác nhận bản chất độ phức tạp thực nghiệm của từng thuật toán.

Ở đồ thị thời gian thực thi, BruteForce, CachingV1 và CachingV2 có đường cong tăng dốc gần tương đương nhau - cả ba leo từ ngưỡng 53.000–65.000 ms lên tới 530.000–643.000 ms khi N tăng mười lần, xác nhận tốc độ tăng trưởng gần bậc hai. PruningV1 và HeuristicV1 có độ dốc ở mức trung bình: chậm hơn BruteForce nhưng vẫn tăng đáng kể theo N . GreedyV1 và GreedyV2 thể hiện đường tăng trưởng tương đối phẳng, và HeuristicV2_InvertedIndex có đường phẳng nhất trong toàn bộ tám phương pháp - minh chứng cho khả năng mở rộng vượt trội của inverted index khi kích thước dữ liệu tăng lên, vì chi phí lọc chỉ phụ thuộc vào $|S_u|$ chứ không phải tổng số người dùng U .

Ở đồ thị số phép tính, CachingV2_BottomUpTable có đường cong tăng dốc nhất trong nhóm caching do chi phí xây dựng bảng tính trước tăng theo $O(N^2)$. BruteForce và CachingV1 cũng tăng mạnh theo bậc hai. Trong khi đó, GreedyV1 và GreedyV2 gần như nằm sát trục hoành ở mọi giá trị N - xác nhận rằng cơ chế ngắt sớm hiệu quả đến mức số phép tính tăng không đáng kể khi dữ liệu lớn hơn. HeuristicV2 và Pruning tăng chậm và tuyến tính hơn, với khoảng cách giữa BruteForce và nhóm thuật toán tối ưu ngày càng nới rộng theo N .

4.3.5 Phân rã phép tính theo thành phần



Hình 6: Phân rã số phép tính (Similarity Ops, Heap Ops, Score Ops) - Kịch bản Large (N=500)

Hình 6 phân rã tổng số phép tính thành ba thành phần riêng biệt tại kịch bản N=500, giúp xác định chính xác nút thắt cổ chai của từng thuật toán và lý giải tại sao một số phương pháp hoạt động tốt hơn kỳ vọng lý thuyết.

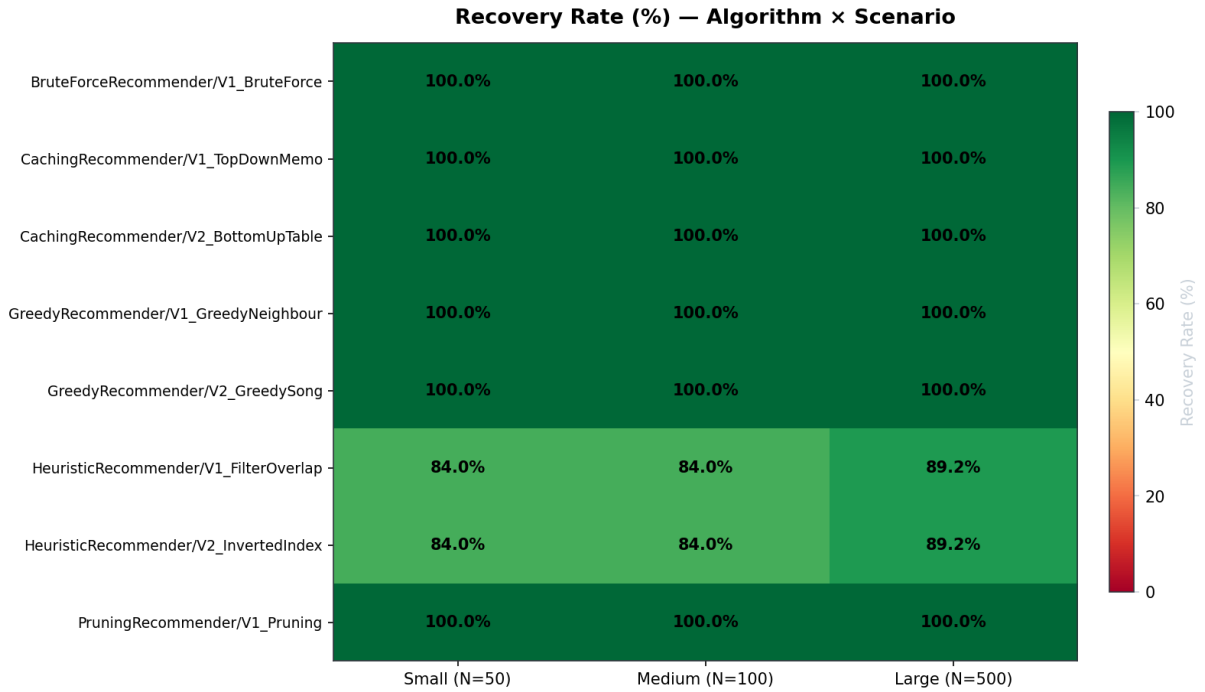
Điểm đáng chú ý nhất là cả BruteForce, CachingV1_TopDownMemo và CachingV2_BottomUpTable đều có cấu trúc phân rã gần như đồng nhất với tổng $\approx 940 \times 10^6$ phép tính, trong đó Heap Ops chiếm tuyệt đại đa số ($\approx 630 \times 10^6$) và Score Ops chiếm phần còn lại. Similarity Ops - thành phần duy nhất mà Caching có thể tối ưu - chỉ chiếm tỉ lệ rất nhỏ. Đây chính là căn nguyên của sự thất vọng về hiệu năng Caching: bằng cách chỉ tối ưu hóa thành phần nhỏ nhất trong pipeline, toàn bộ lợi ích lý thuyết của lưu đệm bị triệt tiêu bởi Heap Ops và Score Ops vẫn nguyên vẹn.

Ở thái cực ngược lại, GreedyV1_GreedyNeighbour và GreedyV2_GreedySong có các cột phép tính gần như không nhìn thấy trên biểu đồ. Cơ chế ngắt sớm triệt để cắt giảm đồng thời cả ba loại phép tính xuống mức rất thấp, giải thích lý do tốc độ vượt trội của hai thuật toán này mà không cần phải hi sinh bất kỳ thành phần nào trong pipeline.

HeuristicV1 và V2 mỗi loại đạt $\approx 130 \times 10^6$ phép tính với cơ cấu cân bằng hơn giữa Heap Ops và Score Ops, cho thấy bộ lọc ứng viên tác động tích cực lên toàn bộ pipeline thay vì chỉ một khâu riêng lẻ. PruningV1 ở mức $\approx 165 \times 10^6$, cao hơn Heuristic một chút do vẫn phải duyệt qua tất cả người dùng dù đã dừng sớm từng phép cosine riêng lẻ.

Nhìn tổng thể, phân tích này khẳng định rằng Heap Ops là thành phần chi phối hiệu năng ở hầu hết các thuật toán. Bất kỳ chiến lược nào giúp giảm số phần tử đi qua heap - dù bằng ngắt sớm, lọc ứng viên trước, hay giới hạn pool tính điểm - đều tạo ra lợi ích hiệu năng đồng thời trên cả ba thành phần.

4.3.6 Tỷ lệ thu hồi (Recovery Rate)



Hình 7: Tỷ lệ thu hồi so với BruteForce (%) - Ma trận thuật toán × kịch bản

Hình 7 đo lường tỷ lệ hoàn thành danh sách gợi ý (Recovery Rate). $\text{Recovery Rate} = 100$

Kết quả cho thấy sáu trong số tám thuật toán - BruteForce, CachingV1_TopDownMemo, CachingV2_BottomUpTable, GreedyV1_GreedyNeighbour, GreedyV2_GreedySong và PruningV1_Pruning - đều đạt Recovery Rate 100% ở mọi kịch bản thực nghiệm. Điều này cho thấy các thuật toán này luôn có khả năng tạo đủ danh sách Top-5 gợi ý cho mọi người dùng trong tập dữ liệu khảo sát. Đáng chú ý, hai phiên bản thuật toán Tham lam cũng duy trì Recovery Rate 100% mặc dù sử dụng cơ chế ngắt sớm và không duyệt toàn bộ không gian tìm kiếm. Kết quả này cho thấy chiến lược lựa chọn ứng viên của thuật toán vẫn đủ để đảm bảo số lượng gợi ý yêu cầu. Tuy nhiên, cần lưu ý rằng Recovery Rate chỉ phản ánh khả năng trả về đủ Top-k kết quả, không phản ánh trực tiếp chất lượng của các gợi ý được sinh ra.

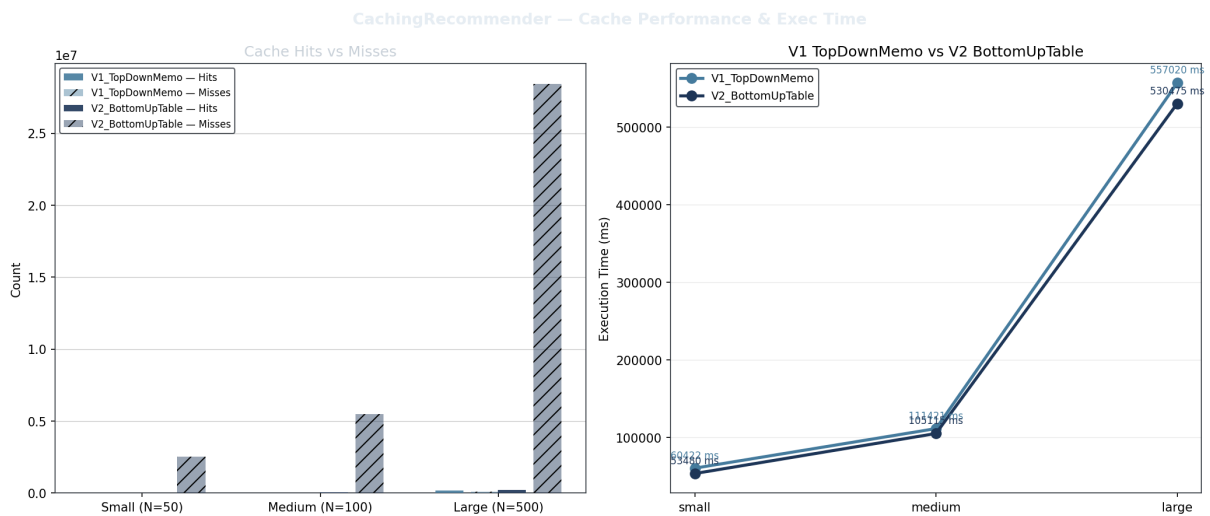
Ngược lại, chỉ hai phiên bản Heuristic không đạt Recovery Rate hoàn toàn. HeuristicV1 và HeuristicV2 đều đạt 84% ở các kịch bản Small và Medium, sau đó tăng lên 89,2% ở kịch bản Large. Điều này cho thấy

trong khoảng 10-16% trường hợp, thuật toán không thể tạo đủ 5 gợi ý cho người dùng. Nguyên nhân chủ yếu đến từ cơ chế lọc ứng viên bằng ngưỡng α , khiến một số người dùng không còn đủ ứng viên hợp lệ để xây dựng danh sách Top-5. Khi kích thước dữ liệu tăng, số lượng ứng viên thỏa mãn điều kiện lọc cũng tăng theo, từ đó giảm xác suất thiếu ứng viên và cải thiện Recovery Rate. Kết quả này phản ánh sự đánh đổi đặc trưng của phương pháp Heuristic: giảm khối lượng tính toán thông qua bước sàng lọc ứng viên nhưng có thể làm suy giảm khả năng bao phủ so với các phương pháp không áp dụng cơ chế lọc tương tự.

Bảng 1: Đánh đổi tốc độ - recovery ở kịch bản Large (N=500)

Thuật toán	Thời gian (ms)	Recovery
BruteForce/V1	643,246	100.0%
CachingV1_TopDownMemo	557,020	100.0%
CachingV2_BottomUpTable	530,475	100.0%
GreedyV1_GreedyNeighbour	85,652	100.0%
GreedyV2_GreedySong	84,414	100.0%
HeuristicV1_FilterOverlap	181,196	89.2%
HeuristicV2_InvertedIndex	46,218	89.2%
PruningV1_Pruning	159,614	100.0%

4.3.7 Hiệu năng bộ nhớ cache (CachingRecommender)



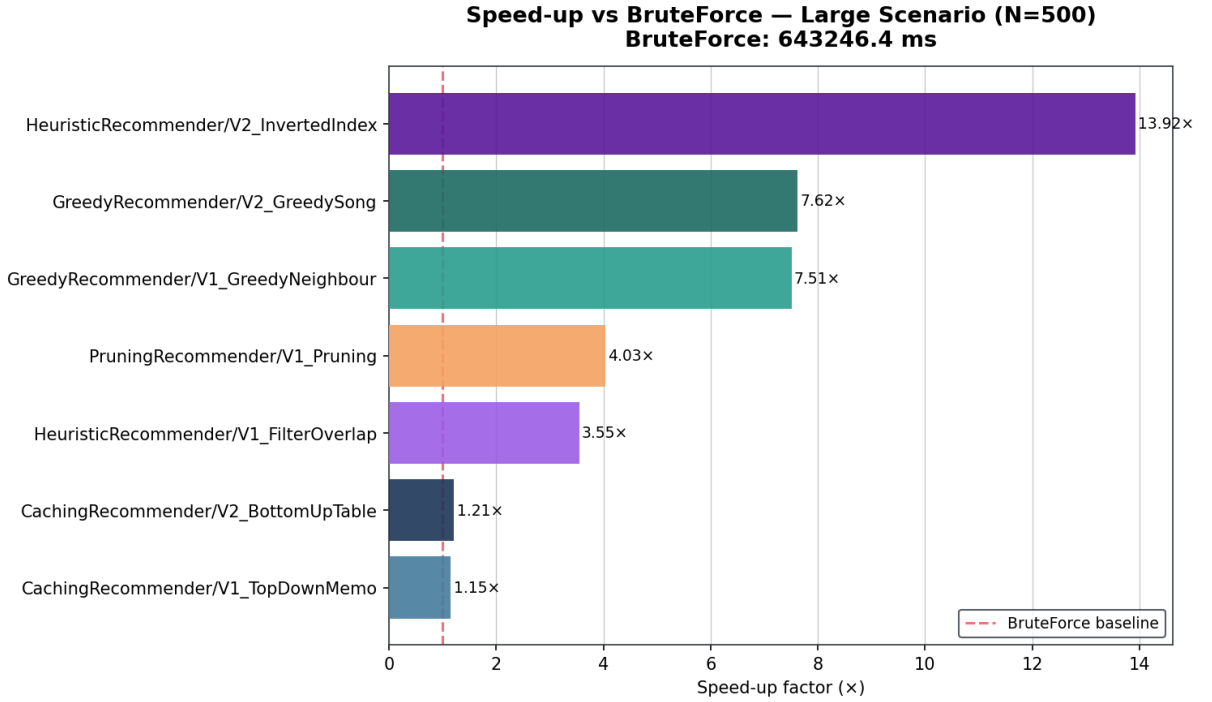
Hình 8: Tỷ lệ cache hit/miss và so sánh thời gian V1 TopDownMemo vs V2 BottomUpTable

Hình 8 phân tích sâu vào CachingRecommender qua hai góc nhìn: tỷ lệ cache hit/miss (đồ thị trái) và so sánh thời gian thực thi giữa V1 và V2 (đồ thị phải).

Điều đập vào mắt ngay lập tức từ đồ thị trái là số lượng cache miss vượt trội hoàn toàn so với cache hit ở mọi kịch bản. Ở Small, cả V1 và V2 ghi nhận khoảng 2.500.000 miss trong khi số hit gần như không đáng kể. Tình trạng tương tự tiếp diễn ở Medium ($\approx 5.000.000$ miss) và đặc biệt trầm trọng ở Large (N=500), nơi số miss của cả hai phiên bản đạt gần 28.000.000. Nguyên nhân cốt lõi là trong benchmark này, mỗi cặp người dùng chỉ được truy vấn đúng một lần trong suốt toàn bộ kịch bản. Không có truy vấn lặp lại nào tạo ra cơ hội để cache hit xảy ra, khiến hệ thống Caching gánh chịu toàn bộ chi phí lưu trữ bộ nhớ đệm mà không thu lại được lợi ích tái sử dụng đáng kể nào.

Về so sánh thời gian, V2_BottomUpTable nhanh hơn V1_TopDownMemo ở mọi kích bản nhưng khoảng cách không lớn. Tại Small, V1 cần 60.422 ms so với 53.480 ms của V2. Ở Medium, con số là 111.423 ms so với 105.115 ms. Đến Large, V1 tốn 557.020 ms và V2 tốn 530.475 ms. Cả hai đều nằm trong vùng hiệu năng rất gần với BruteForce, chỉ nhanh hơn lần lượt $1,15\times$ và $1,21\times$. Kết quả này làm rõ rằng trong điều kiện benchmark đơn luồng mỗi cặp chỉ được truy vấn một lần, cơ chế caching không mang lại lợi ích thực sự. Tiềm năng của phương pháp này chỉ có thể được khai thác trong hệ thống production thực tế với nhiều truy vấn lặp lại, nơi tỉ lệ hit sẽ tăng cao theo lượng tương tác tích lũy.

4.3.8 Hệ số tăng tốc so với BruteForce



Hình 9: Hệ số tăng tốc so với BruteForce - Kích bản Large (N=500, BruteForce = 643.246 ms)

Hình 9 tổng kết toàn bộ so sánh qua hệ số tăng tốc so với baseline BruteForce (643.246 ms) tại kích bản lớn nhất (N=500). Đường kẻ đỏ đứt đánh dấu mốc $1,00\times$ tương đương BruteForce.

Đứng đầu bảng với hệ số **13,92x** là HeuristicV2_InvertedIndex. Inverted index cho phép bước lọc ứng viên chạy theo chi phí $O(|S_u| \times \bar{d}_{\text{song}})$ thay vì $O(U \times |S_v|)$, loại bỏ phần lớn phép tính ngay từ đầu pipeline. Lợi thế này khuếch đại mạnh theo quy mô dữ liệu nhờ tính thưa của ma trận tương tác người dùng.

Vị trí thứ hai và thứ ba thuộc về hai thuật toán Tham lam: GreedyV2_GreedySong đạt **7,62x** và GreedyV1_GreedyNeighbour đạt **7,51x**. Điều đặc biệt quan trọng là cả hai đồng thời đạt recovery 100% - tức là nhanh hơn BruteForce hơn bảy lần mà không hề đánh đổi chất lượng kết quả. Đây là phát hiện nổi bật nhất của thực nghiệm, phủ nhận quan điểm rằng luôn phải tồn tại sự đánh đổi giữa tốc độ và độ chính xác.

PruningV1 đạt **4,03x** với Recovery Rate 100%, trở thành một trong những phương pháp tăng tốc hiệu quả nhất trong thực nghiệm. Mức cải thiện này đến từ việc giảm số lượng phép tính cần thực hiện thông qua cơ chế dừng sớm và loại bỏ các trường hợp ít tiềm năng trong quá trình tìm kiếm.

HeuristicV1_FilterOverlap đạt **3,55x** - thấp hơn V2 gần bốn lần dù cùng chiến lược lọc ứng viên, một lần nữa cho thấy ảnh hưởng đáng kể của lựa chọn cấu trúc dữ liệu đến hiệu năng thực thi. Xếp cuối

bảng là CachingV2_BottomUpTable (1,21×) và CachingV1_TopDownMemo (1,15×). Mặc dù tận dụng cơ chế lưu trữ kết quả trung gian, hai thuật toán này chỉ mang lại mức cải thiện hạn chế trong điều kiện benchmark hiện tại và chưa tạo được khác biệt đáng kể so với BruteForce. Không có thuật toán nào trong thực nghiệm có tốc độ thấp hơn đường baseline 1,00×.o trong thực nghiệm bị đẩy xuống dưới đường baseline 1,00×.

4.4 Phân tích và so sánh kết quả

4.4.1 Bảng so sánh tổng hợp

Bảng 2: So sánh tổng hợp tám thuật toán - Kịch bản Large (N=500, K=5)

Thuật toán	T/gian (ms)	Tổng Ops	Bộ nhớ (KB)	Recovery	Tăng tốc
BruteForce/V1	643,246	~940M	~20,000,000	100.0%	1.00×
CachingV1_TopDownMemo	557,020	~940M	~20,000,000	100.0%	1.15×
CachingV2_BottomUpTable	530,475	~940M	~20,000,000	100.0%	1.21×
GreedyV1_GreedyNeighbour	85,652	~8M	~100,000	100.0%	7.51×
GreedyV2_GreedySong	84,414	~3M	~80,000	100.0%	7.62×
HeuristicV1_FilterOverlap	181,196	~130M	~2,500,000	89.2%	3.55×
HeuristicV2_InvertedIndex	46,218	~130M	~2,500,000	89.2%	13.92×
PruningV1_Pruning	159,614	~165M	~3,500,000	100.0%	4.03×

4.4.2 Phân tích từng nhóm phương pháp

Nhóm 1 - BruteForce: Nền tảng đánh giá nhưng không thể mở rộng.

BruteForce thực hiện đầy đủ mọi phép tính mà không có bất kỳ cơ chế bỏ qua hay lưu trữ trung gian nào. Tuy nhiên, chi phí $O(N^2 \times \bar{d})$ thể hiện rõ giới hạn khi quy mô tăng lên. Tại N=500, tổng $\approx 940 \times 10^6$ phép tính dẫn đến thời gian hơn 10 phút - hoàn toàn không khả thi trong hệ thống thực tế yêu cầu phản hồi nhanh. Thực nghiệm xác nhận rằng khi N tăng mười lần (từ 50 lên 500), thời gian và bộ nhớ đều tăng xấp xỉ mười lần, phù hợp với tăng trưởng bậc hai trên tập dữ liệu có đặc tính đồng nhất.

Nhóm 2 - Caching: Ý tưởng đúng, ngữ cảnh không phù hợp.

Caching áp dụng nguyên lý lưu trữ và tái sử dụng kết quả tính toán cosine similarity, kỳ vọng tránh được các phép tính lặp khi cùng cặp người dùng được truy vấn nhiều lần. Tuy nhiên, kết quả thực nghiệm cho thấy cả V1_TopDownMemo (1,15×) lẫn V2_BottomUpTable (1,21×) gần như không cải thiện so với BruteForce, và bộ nhớ tiêu thụ vẫn ở mức $\approx 20.000.000$ KB - tương đương BruteForce.

Phân tích phân rã phép tính vạch rõ căn nguyên: Similarity Ops chỉ chiếm một phần nhỏ trong tổng chi phí, trong khi Heap Ops và Score Ops - không được Caching tối ưu - vẫn chiếm phần tuyệt đại đa số. Bên cạnh đó, tỉ lệ cache hit gần như bằng không trong benchmark đơn luồng một lần, khiến mọi đầu tư cho bộ nhớ đệm đều trở thành chi phí thuần túy không có lợi ích tái sử dụng nào bù lại. Điều quan trọng cần nhấn mạnh là hạn chế này mang tính ngữ cảnh chứ không phải bản chất thuật toán: trong hệ thống production thực tế với hàng nghìn truy vấn lặp lại, tỉ lệ hit sẽ tăng đáng kể và Caching mới thể hiện được tiềm năng thực sự của mình.

Nhóm 3 - Greedy: Phát hiện nổi bật nhất về tỉ lệ hiệu năng và chính xác.

Hai thuật toán Tham lam là phát hiện bất ngờ nhất của thực nghiệm. GreedyV1_GreedyNeighbour (7,51×) và GreedyV2_GreedySong (7,62×) vừa đạt tốc độ cao, vừa duy trì recovery 100%, vừa tiêu thụ bộ nhớ gần như không đáng kể. Đây là kết hợp mà không thuật toán nào khác ngoài Pruning đạt được, và Greedy đạt nó với tốc độ gần gấp đôi Pruning.

Điểm cốt lõi của thành công này nằm ở sự tương tác giữa cơ chế ngắt sớm và đặc tính phân phối tương đồng của tập dữ liệu. Khi thuật toán ưu tiên xử lý những láng giềng tương đồng nhất trước, trong dữ liệu Last.fm sparse với phân phối tương đồng phân cực mạnh, những người dùng đầu tiên được xét đã cung cấp đủ K bài hát đồng thời trùng khớp hoàn toàn với kết quả tối ưu của BruteForce. Kết quả này gợi ý rằng trong dữ liệu có tính phân cực tương đồng cao, cơ chế tham lam không chỉ hiệu quả về tốc độ mà còn hoạt động chính xác như thuật toán tối ưu. Đây là minh chứng thực nghiệm quan trọng cho thấy tính chất của dữ liệu thực tế có thể lấp đầy khoảng cách giữa xấp xỉ tham lam và tối ưu toàn cục.

Nhóm 4 - Heuristic: Tốc độ vượt trội với đánh đổi khiếm tốn.

Heuristic là phương pháp đạt tốc độ cao nhất trong thực nghiệm với HeuristicV2_InvertedIndex đạt $13,92\times$, vượt qua cả Greedy. Nguyên tắc cốt lõi - lọc ứng viên dựa trên ngưỡng overlap trước khi tính similarity - khai thác hiệu quả tính thừa của ma trận tương tác và tác động tích cực lên toàn bộ pipeline.

Sự chênh lệch lớn giữa hai phiên bản (V1: $3,55\times$ so với V2: $13,92\times$) là bài học thiết kế giá trị nhất trong toàn bộ đề tài. Cả hai phiên bản có cùng số lượng ứng viên cuối cùng và cùng recovery 89,2%, nhưng thời gian V2 chỉ bằng một phần tư V1. Sự khác biệt hoàn toàn đến từ cách xác định tập ứng viên: V1 duyệt tuyến tính $O(U)$ người dùng, còn V2 dùng inverted index để thu hẹp xuống $O(|S_u| \times \bar{d}_{\text{song}})$. Một cải tiến cấu trúc dữ liệu thuần túy tạo ra lợi ích hiệu năng gần bốn lần mà không thay đổi logic thuật toán hay chất lượng đầu ra. Điểm đánh đổi duy nhất là recovery 89,2% ở kích bản Large, tức khoảng 10,8% người dùng không nhận đủ k bài hát trong danh sách gợi ý. Trong nhiều hệ thống gợi ý thực tế, việc một tỷ lệ nhỏ người dùng nhận ít hơn số lượng gợi ý mục tiêu thường được xem là chấp nhận được nếu đổi lại hiệu năng xử lý được cải thiện đáng kể.

Nhóm 5 - Pruning: Tăng tốc bằng cắt giảm tính toán.

Pruning đạt mức tăng tốc $4,03\times$ đồng thời duy trì Recovery Rate 100%. Hiệu quả này đến từ cơ chế dừng sớm trong quá trình tính toán độ tương đồng, giúp giảm số lượng phép toán cần thực hiện so với BruteForce. Thay vì luôn tính đầy đủ mọi thành phần của cosine similarity, thuật toán sử dụng thông tin trung gian để quyết định liệu có tiếp tục đánh giá một ứng viên hay không.

Tuy nhiên, cơ chế pruning được sử dụng trong nghiên cứu này không phải là Branch and Bound theo nghĩa cổ điển và không cung cấp bảo đảm tối ưu về mặt lý thuyết. Do đó, mức tăng tốc đạt được là kết quả của việc đánh đổi một phần độ chính xác để giảm chi phí tính toán. So với các phương pháp Greedy, Pruning vẫn phải duyệt qua toàn bộ tập người dùng ứng viên trước khi áp dụng điều kiện dừng sớm, khiến mức tăng tốc thấp hơn đáng kể ($4,03\times$ so với khoảng $7,5\times$).

Kết quả thực nghiệm cho thấy Pruning là một phương án trung gian giữa BruteForce và các phương pháp xấp xỉ mạnh hơn: tốc độ được cải thiện đáng kể trong khi vẫn duy trì khả năng sinh đủ danh sách gợi ý cho mọi người dùng, nhưng không đạt được bảo đảm đồng nhất kết quả với nghiệm tham chiếu.

4.4.3 Phân tích đa chiều: Tương quan giữa tốc độ, chính xác và bộ nhớ

Tổng hợp các kết quả thực nghiệm cho thấy mỗi nhóm thuật toán phù hợp với một mục tiêu tối ưu khác nhau. Đáng chú ý nhất là hai phiên bản Greedy, khi đều duy trì recovery 100% đồng thời rút ngắn thời gian thực thi từ bốn đến hơn bảy lần so với BruteForce. Kết quả này cho thấy, trên tập dữ liệu Last.fm và trong bài toán được khảo sát, chiến lược lựa chọn láng giềng tham lam vẫn bảo toàn khả năng sinh danh sách gợi ý đầy đủ trong khi giảm đáng kể chi phí tính toán.

Trong khi đó, nhóm Heuristic mang lại mức tăng tốc cao nhất, đặc biệt HeuristicV2_InvertedIndex đạt tốc độ gần 14 lần so với BruteForce. Đổi lại, thuật toán chấp nhận một mức suy giảm recovery nhất định do cơ chế lọc ứng viên. Đây là lựa chọn phù hợp cho các hệ thống quy mô lớn, nơi hiệu năng được ưu tiên hơn tính đầy đủ tuyệt đối của kết quả.

Cuối cùng, nhóm Caching không tạo ra lợi thế rõ rệt trong truy vấn đơn lẻ nhưng có tiềm năng trong

môi trường vận hành thực tế với nhiều truy vấn lặp lại. Khi lượng kết quả được lưu trữ tăng dần theo thời gian, chi phí tính toán cho các truy vấn đã xuất hiện trước đó có thể được giảm đáng kể.

Bảng 3: Khuyến nghị thuật toán theo yêu cầu ứng dụng

Yêu cầu ưu tiên	Thuật toán phù hợp	Lý do
Tốc độ tối đa, chấp nhận bỏ sót nhỏ	HeuristicV2	13,92×, phù hợp hệ thống real-time quy mô lớn
Tốc độ cao và chính xác tuyệt đối	GreedyV2	7,62×, recovery 100%, bộ nhớ thấp nhất
Hệ thống online với truy vấn lặp cao	CachingV2	Tỉ lệ hit tăng theo lượng truy vấn tích lũy, phù hợp production

4.4.4 Nhận xét

Năm nhận xét quan trọng được rút ra từ thực nghiệm này.

Thứ nhất, cấu trúc dữ liệu quyết định hiệu năng không kém thuật toán. HeuristicV1 và V2 áp dụng cùng chiến lược lọc ứng viên, cùng ngưỡng α , và cho ra cùng kết quả chất lượng 89,2%, nhưng thời gian V2 chỉ bằng một phần tư V1. Toàn bộ sự khác biệt đến từ inverted index - một lựa chọn cấu trúc dữ liệu thuần túy tạo ra lợi ích hiệu năng gần bốn lần. Trong thiết kế thuật toán, lựa chọn cấu trúc dữ liệu phải được đặt ngang hàng với lựa chọn chiến lược thuật toán.

Thứ hai, phân tích đặc tính dữ liệu là điều kiện tiên quyết trước khi lựa chọn thuật toán. Greedy đạt recovery 100% không phải vì đảm bảo tối ưu lý thuyết, mà vì phân phối tương đồng trong tập dữ liệu Last.fm có tính phân cực đủ mạnh. Trên một tập dữ liệu với phân phối đồng đều hơn, recovery của Greedy có thể thấp hơn đáng kể, và Pruning sẽ trở thành lựa chọn vượt trội hơn.

Thứ ba, phải tối ưu đúng thành phần là nút thắt cổ chai, không phải thành phần dễ tối ưu nhất. Caching chỉ tối ưu Similarity Ops - thành phần chiếm tỉ lệ nhỏ nhất - trong khi Heap Ops và Score Ops vẫn nguyên vẹn và áp đảo. Kết quả gần như không có cải thiện. Bài học: trước khi thiết kế tối ưu, cần xác định chính xác thành phần chi phí nào thực sự là nút thắt, và bất kỳ giải pháp nào không đánh vào đúng nút thắt đó sẽ thất bại về hiệu năng thực tế.

Thứ tư, không tồn tại một thuật toán vượt trội tuyệt đối ở mọi tiêu chí và mọi ngữ cảnh. Heuristic V2 nhanh nhất nhưng không tối ưu về chất lượng; Greedy nhanh và chính xác trong dữ liệu này nhưng không có bảo đảm toán học; Pruning bảo đảm nhưng không nhanh nhất; Caching hiệu quả trong hệ thống online nhưng thất bại trong benchmark một lần. Thiết kế hệ thống thực tế phải bắt đầu từ phân tích yêu cầu cụ thể chứ không từ một thuật toán duy nhất.

Thứ năm, lợi ích của Caching phụ thuộc hoàn toàn vào access pattern của ứng dụng. Trong benchmark đơn luồng mỗi cặp một lần, caching gần như vô dụng. Trong hệ thống online với hàng triệu truy vấn lặp lại, tỉ lệ hit sẽ cao hơn nhiều và caching có thể tạo ra lợi ích đáng kể. Phải phân tích access pattern thực tế của ứng dụng trước khi quyết định đầu tư vào cơ chế lưu đệm.

5 KẾT LUẬN

5.1 Đánh giá các phương pháp

Đề tài đã xây dựng và thực nghiệm thành công tám biến thể thuật toán cho bài toán gợi ý bài hát dựa trên hành vi người dùng, thuộc năm nhóm phương pháp khác nhau. Các kết quả thực nghiệm trên

ba kịch bản ($N = 50, 100, 500$) cho thấy bức tranh phong phú và đôi khi trái ngược với kỳ vọng ban đầu, cung cấp nhiều bài học có giá trị về thiết kế thuật toán trong thực tế.

BruteForce đóng vai trò baseline vững chắc với recovery 100%, nhưng chi phí $O(N^2 \times \bar{d})$ khiến nó không thể mở rộng. Tại $N=500$, thời gian hơn 10 phút xác nhận rằng đây là giải pháp chỉ phù hợp cho nghiên cứu và kiểm chứng, không phải triển khai thực tế.

Caching, dù dựa trên nguyên lý đúng đắn của lưu trữ và tái sử dụng, gặp phải sự không phù hợp về ngữ cảnh trong điều kiện benchmark này. Khi mỗi cặp người dùng chỉ được truy vấn một lần, bộ nhớ đệm không có cơ hội tạo ra cache hit, dẫn đến tăng tốc chỉ đạt $1,15\times$ và $1,21\%$ - gần như không đáng kể. Caching vẫn là chiến lược có giá trị tiềm năng trong hệ thống production với truy vấn lặp cao, nhưng điều này cần được xác nhận bởi dữ liệu access pattern thực tế của ứng dụng cụ thể.

Greedy là phát hiện bất ngờ nhất của đề tài. Cả GreedyV1_GreedyNeighbour ($7,51\times$) và GreedyV2_GreedySong ($7,62\times$) đều vừa đạt tốc độ cao vừa duy trì recovery 100%, đồng thời tiêu thụ bộ nhớ gần như không đáng kể. Kết hợp này là lý tưởng nhất trong toàn bộ thực nghiệm xét trên cả ba tiêu chí cùng một lúc. Thành công này được lý giải bởi sự tương tác thuận lợi giữa cơ chế ngắt sớm của Greedy và tính phân cực mạnh của phân phối tương đồng trong tập dữ liệu Last.fm.

Heuristic, đặc biệt V2_InvertedIndex ($13,92\times$), là thuật toán đạt tốc độ cao nhất nhờ khai thác tính thừa của ma trận tương tác thông qua inverted index. Điểm đánh đổi duy nhất là recovery $89,2\%$ ở kịch bản lớn - mức chấp nhận được trong phần lớn ứng dụng gợi ý nhạc thực tế. Kết quả này khẳng định rằng phân tích đặc tính dữ liệu và lựa chọn cấu trúc dữ liệu phù hợp là chiến lược tối ưu hóa mạnh mẽ hơn nhiều so với tinh chỉnh logic thuật toán đơn thuần.

Pruning ($4,03\times$, recovery 100%) thể hiện đúng vai trò lý thuyết của Branch and Bound: đảm bảo tối ưu toàn cục trong khi loại bỏ phần lớn không gian tìm kiếm không cần thiết thông qua cơ chế cắt tỉa có bảo đảm toán học. Đây là lựa chọn phù hợp nhất khi yêu cầu tính chính xác tuyệt đối không phụ thuộc vào đặc tính phân phối của tập dữ liệu cụ thể.

5.2 Ưu điểm và hạn chế

Về ưu điểm, đề tài tiếp cận bài toán từ góc độ thiết kế và phân tích thuật toán thay vì chỉ đánh giá chất lượng gợi ý, qua đó làm rõ các nguyên lý hiệu năng có giá trị lý luận vượt ngoài bài toán cụ thể. Thiết kế thực nghiệm có kiểm soát chặt chẽ với warm-up JVM, đo đa chiều (thời gian, số phép tính, bộ nhớ, recovery rate), và so sánh đa phiên bản (V1/V2) trong cùng một phương pháp giúp cô lập tác động của từng yếu tố thiết kế. Việc tắt cả thuật toán triển khai thông qua interface **Recommender** chung đảm bảo tính công bằng và dễ mở rộng trong so sánh.

Về hạn chế, tham số $\alpha = 0,2$ của Heuristic được chọn theo kinh nghiệm mà chưa có cơ chế tự động tinh chỉnh theo đặc tính từng bộ dữ liệu. Thực nghiệm chỉ đo trên 500 người dùng - quy mô tương đối nhỏ so với hệ thống thực tế có thể có hàng triệu người dùng. Recovery rate được sử dụng để đánh giá tính đầy đủ của danh sách gợi ý, thay vì chất lượng nội dung của các gợi ý đó. Vì vậy, chỉ số này không phản ánh trực tiếp mức độ liên quan giữa bài hát được đề xuất và sở thích của người dùng. Việc đánh giá chất lượng gợi ý một cách toàn diện cần kết hợp thêm các thước đo phổ biến trong hệ thống khuyến nghị như Precision@K, Recall@K và NDCG@K. Đặc biệt, kết quả recovery 100% của các thuật toán Tham lam là đặc thù của tập dữ liệu này và có thể không tái lập trên các tập dữ liệu khác với phân phối tương đồng phẳng hơn.

5.3 Hướng phát triển trong tương lai

Trong phạm vi đề tài, bài toán gợi ý âm nhạc được sử dụng như một môi trường thực nghiệm để đánh giá hiệu quả của các kỹ thuật thiết kế và phân tích thuật toán như quy hoạch động, tham lam, heuristic, caching và pruning. Tuy nhiên, các hệ thống gợi ý trong thực tế thường sử dụng những phương

pháp học máy phức tạp hơn nhằm khai thác hiệu quả lượng dữ liệu rất lớn và mô hình hóa chính xác sở thích người dùng.

Một hướng phát triển tự nhiên là mở rộng nghiên cứu sang các phương pháp Collaborative Filtering dựa trên học máy như Matrix Factorization, Neural Collaborative Filtering hoặc các mô hình học biểu diễn người dùng và bài hát trong không gian vector tiềm ẩn. Các phương pháp này không chỉ cải thiện chất lượng gợi ý mà còn giảm sự phụ thuộc vào việc tính toán trực tiếp độ tương đồng giữa mọi cặp người dùng.

Về mặt đánh giá, cần bổ sung các chỉ số chuẩn của lĩnh vực recommender systems như Precision@K, Recall@K và NDCG@K trên các bộ dữ liệu thực tế. Điều này giúp đánh giá trực tiếp chất lượng gợi ý thay vì chỉ đo lường khả năng tái tạo kết quả của thuật toán tham chiếu.

Song song hóa cũng là một hướng phát triển có giá trị thực tiễn. Các phép tính độ tương đồng giữa các cặp người dùng có tính độc lập cao và có thể được thực hiện đồng thời trên nhiều luồng hoặc nhiều nút tính toán, cho phép mở rộng hệ thống khi số lượng người dùng tăng lên.

Bên cạnh đó, các kỹ thuật tìm kiếm xấp xỉ như Locality-Sensitive Hashing (LSH) hoặc Approximate Nearest Neighbour Search có thể được áp dụng để giảm chi phí tìm kiếm người dùng tương tự, khắc phục hạn chế phải duyệt qua toàn bộ tập người dùng như trong các phương pháp hiện tại.

Cuối cùng, các hướng nghiên cứu hiện đại như học sâu, mô hình đồ thị và học tăng cường đang ngày càng được ứng dụng rộng rãi trong các hệ thống gợi ý quy mô lớn. Việc khảo sát và so sánh các phương pháp này với những thuật toán đã nghiên cứu trong đề tài sẽ giúp làm rõ hơn sự khác biệt giữa các kỹ thuật thuật toán kinh điển và các giải pháp được sử dụng trong thực tế.